



Tribhuvan University

Faculty of Humanities and Social Sciences

A PROJECT REPORT

ON

Smart Helpdesk

A Multi-Tenant Support Ticket Management System

Submitted to

Department of BCA

Everest College

Thapathali, Kathmandu

In partial fulfillment of the requirements for the Bachelor in Computer Application
(CACS452 Project III)

Submitted by

Subham Bhandari, 6-2-355-29-2021

Utsav Karki, 6-2-355-33-2021

BCA 8th Semester, IV Year

Under the Supervision of

Mr. Kamal Pathak



Tribhuvan University

Faculty of Humanities and Social Sciences

Everest College

Supervisor's Recommendation

I hereby recommend that this project prepared under my supervision by SUBHAM BHANDARI, 6-2-355-29-2021 and UTSAV KARKI, 6-2-355-33-2021 entitled "Smart Helpdesk: A Multi-Tenant Support Ticket Management System" in partial fulfillment of the requirements for the degree of Bachelor in Computer Application is recommended for the final evaluation.

.....
Mr. Kamal Pathak

Supervisor

Department of BCA

Everest College, Thapathali, Kathmandu



Tribhuvan University

Faculty of Humanities and Social Sciences

Everest College

Letter of Approval

This is to certify that this project prepared by SUBHAM BHANDARI, 6-2-355-29-2021 and UTSAV KARKI, 6-2-355-33-2021 entitled "Smart Helpdesk: A Multi-Tenant Support Ticket Management System" in partial fulfillment of the requirements for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

.....
SUPERVISOR

Mr. Kamal Pathak
Department of BCA
Everest College

.....
HOD / CO-ORDINATOR

Mr. Kamal Pathak
Department of BCA
Everest College

.....
INTERNAL EXAMINER

.....
EXTERNAL EXAMINER

Acknowledgement

We are glad to present this report on "Smart Helpdesk", prepared as our CACS452 Project III in the eighth semester of the Bachelor in Computer Application. We express our sincere gratitude to everyone who guided and supported us throughout the project.

We extend our genuine thanks to our project supervisor, Mr. Kamal Pathak, for his continuous guidance, constructive feedback and encouragement from the proposal to the final defence. We are equally thankful to Mr. Kamal Pathak, Head of the Department of BCA, for his support and for the academic environment in which this work was possible.

We are indebted to the Department of BCA, Everest College, and to the faculty members whose courses in database management, software engineering, web technology, operating systems, data mining and research methodology laid the foundation of this project. We also thank our families and friends for their patience and support, and the open-source communities whose frameworks, databases and tools allowed us to focus our own work on the system's design, its algorithms and its verification.

Abstract

Small support teams often handle requests through shared mailboxes or basic ticketing tools in which priority is set by hand, work is assigned by habit, one outage produces many unrelated tickets and service-level breaches are noticed only after they happen. This project designs and implements **Smart Helpdesk**, a multi-tenant support ticket management system that one installation offers to many independent organisations, either as a hosted service or on an organisation's own server. It is built with a Laravel application programming interface, a React single-page application and PostgreSQL, packaged with Docker and deployed to a cloud virtual machine.

Four operational decisions are made by the system's own simple algorithms, and each decision is explained to the agent: a weighted priority score over impact, urgency, customer tier and waiting time; assignment to the eligible agent with the lowest load relative to capacity; duplicate detection by comparing the word sets of tickets; and a service-level agreement (SLA) timer that pauses, warns and records breaches on working hours. A reporting module records every change to the main records, shows any record as it was at a past moment and analyses backlog and workload over time. Each organisation's data is kept separate in the application and in the database. The system also provides a media library, email notifications, email-to-ticket, a REST API with signed webhooks and a management dashboard.

Manual testing of the main workflows and of each algorithm with worked examples confirmed that the system behaves as specified.

Keywords: helpdesk, multi-tenancy, ticket prioritisation, agent assignment, duplicate detection, service level agreement

Table of Contents

Supervisor's Recommendation.....	i
Letter of Approval.....	ii
Acknowledgement.....	iii
Abstract.....	iv
List of Figures.....	vii
List of Tables.....	viii
List of Abbreviations.....	ix
Chapter 1: Introduction.....	1
1.1 Introduction.....	1
1.2 Problem Statement.....	1
1.3 Objectives.....	2
1.4 Scope and Limitation.....	2
1.5 Development Methodology.....	3
1.6 Report Organization.....	4
Chapter 2: Background Study and Literature Review.....	5
2.1 Background Study.....	5
2.1.1 Tickets, priority and service levels.....	5
2.1.2 Weighted scoring and load balancing.....	5
2.1.3 Text similarity.....	5
2.1.4 Multi-tenant software.....	6
2.2 Literature Review.....	6
2.2.1 Existing helpdesk systems.....	6
2.2.2 Related research.....	7
Chapter 3: System Analysis and Design.....	8
3.1 System Analysis.....	8
3.1.1 Requirement Analysis.....	8
3.1.2 Feasibility Analysis.....	11
3.1.3 Object Modelling using Class and Object Diagrams.....	12
3.1.4 Dynamic Modelling using State and Sequence Diagrams.....	14
3.1.5 Process Modelling using Activity Diagrams.....	18
3.2 System Design.....	22
3.2.1 Refinement of Class, Object, State, Sequence and Activity Diagrams.....	22
3.2.2 Component Diagrams.....	24
3.2.3 Deployment Diagrams.....	25
3.3 Algorithm Details.....	26
3.3.1 Priority scoring: basic weighted priority.....	26
3.3.2 Agent assignment: least-loaded eligible agent.....	27
3.3.3 Duplicate detection: Jaccard word overlap.....	28
3.3.4 SLA evaluation: simple SLA timer.....	28
3.3.5 History reconstruction and time analytics.....	29

Chapter 4: Implementation and Testing.....	30
4.1 Implementation.....	30
4.1.1 Tools Used.....	30
4.1.2 Implementation Details of Modules.....	30
4.2 Testing.....	34
4.2.1 Test Cases for Unit Testing.....	34
4.2.2 Test Cases for System Testing.....	35
4.3 Result Analysis.....	35
Chapter 5: Conclusion and Future Recommendations	37
5.1 Conclusion.....	37
5.2 Future Recommendations.....	37
References.....	38
Appendices	39
Appendix A: Screenshots.....	39

List of Figures

Figure 1.1: Waterfall model of the development.....	3
Figure 3.1: Use case diagram of ticket handling and automation.....	8
Figure 3.2: Use case diagram of administration, contacts and reporting.....	9
Figure 3.3: Gantt chart of the project schedule.....	12
Figure 3.4: Class diagram of the domain model.....	13
Figure 3.5: Object diagram of a seeded ticket.....	14
Figure 3.6: State diagram of the ticket lifecycle.....	15
Figure 3.7: State diagram of an SLA timer.....	16
Figure 3.8: Sequence diagram of ticket creation.....	17
Figure 3.9: Sequence diagram of the SLA check.....	17
Figure 3.10: Sequence diagram of inbound email processing.....	18
Figure 3.11: Activity diagram of agent assignment.....	19
Figure 3.12: Activity diagram of duplicate detection.....	20
Figure 3.13: Activity diagram of inbound email routing.....	21
Figure 3.14: Algorithm interfaces and their baseline implementations.....	23
Figure 3.15: Entity relationship diagram.....	24
Figure 3.16: Component diagram.....	25
Figure 3.17: Deployment diagram.....	26
Figure 4.1: Management dashboard.....	32
Figure 4.2: Ticket screen with conversation, SLA timers and details.....	33
Figure 4.3: Priority explanation shown on a ticket.....	33
Figure A.1: Landing page.....	39
Figure A.2: Sign-in page of a workspace.....	39
Figure A.3: Ticket list with filters.....	40
Figure A.4: New ticket with duplicate suggestions.....	40
Figure A.5: Assignment ranking with excluded agents.....	41
Figure A.6: Ticket close to its SLA deadline.....	41
Figure A.7: Ticket history with the point-in-time view.....	42
Figure A.8: Contact list.....	42
Figure A.9: Report catalogue.....	43
Figure A.10: Priority settings.....	43
Figure A.11: SLA policies.....	44
Figure A.12: Ticket list in the dark theme.....	44
Figure A.13: Platform console with the list of workspaces.....	45

List of Tables

Table 1.1: Phases of the waterfall model and their outputs.....	4
Table 2.1: Comparison of existing open-source helpdesk systems.....	6
Table 3.1: Functional requirements by module.....	9
Table 3.2: Description of use case "Create ticket"	10
Table 3.3: Description of use case "Assign ticket"	10
Table 3.4: Non-functional requirements	11
Table 3.5: Project schedule	11
Table 3.6: Inputs of the priority score.....	27
Table 4.1: Tools and technologies.....	30
Table 4.2: Unit test cases.....	34
Table 4.3: System test cases.....	35

List of Abbreviations

API	Application Programming Interface
BCA	Bachelor in Computer Application
CPU	Central Processing Unit
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
GB	Gigabyte
HOD	Head of Department
HTTPS	Hypertext Transfer Protocol Secure
ID	Identifier
ITIL	Information Technology Infrastructure Library
MB	Megabyte
P1 to P4	Priority levels 1 (critical) to 4 (low)
PHP	PHP: Hypertext Preprocessor
PNG	Portable Network Graphics
REST	Representational State Transfer
SLA	Service Level Agreement
ST, UT	System test, unit test (test case identifiers)
XLSX	Excel spreadsheet file format

Chapter 1: Introduction

1.1 Introduction

Every organisation that serves customers or employees receives requests: a login that fails, an invoice that is wrong, a server that is down. A helpdesk turns these requests into **tickets** that can be prioritised, assigned, tracked against a promised response time and closed with a record of what was done. Commercial platforms such as Zendesk, ServiceNow and Jira Service Management provide this for large enterprises, while small and medium teams often rely on shared mailboxes or open-source tools that store tickets well but leave most decisions to people.

Smart Helpdesk is a web-based support ticket management system built for such teams. One installation serves many independent organisations, called **tenants**, each with its own users, agents, customers, tickets and settings, and the same software can run as a hosted service or on an organisation's own server. Beyond storing tickets, the system makes four decisions automatically and explains each one: how urgent a ticket is, which agent should handle it, whether it repeats an existing ticket, and whether its service-level agreement (SLA) is at risk. It also records how every record changed over time, offers reports on tickets, customers, agents and service levels, sends notifications by email and in the application, turns customer emails into tickets, and lets other systems integrate through a documented API and webhooks.

1.2 Problem Statement

A review of how small support teams work and of existing open-source helpdesks (Section 2.2) shows the following problems:

1. **Inconsistent prioritisation.** Priority is chosen by whoever creates the ticket, and a ticket's urgency does not grow while it waits.
2. **Unbalanced assignment.** Tickets are assigned by hand or by simple rotation, so work piles up on experienced agents while skills and current workload are ignored.
3. **Duplicate tickets.** One outage produces many tickets that describe the same problem in different words, and agents work on them in parallel.
4. **Late SLA awareness.** Response and resolution targets are checked in reports after the fact instead of warning agents before a breach.

5. **Shallow reporting.** Managers see current counts but not how the backlog or a customer's record looked on a past date.
6. **Isolation and deployment.** A hosted helpdesk must guarantee that one organisation never sees another's data, while some organisations need the same software on their own servers.

1.3 Objectives

The objectives of the project are:

1. To design and implement a multi-tenant helpdesk in which each organisation's data is strictly isolated, and which can run on any virtual machine or on an organisation's own server.
2. To implement simple, explainable algorithms for **priority scoring**, **agent assignment**, **duplicate ticket detection** and **SLA monitoring** with working-hours calendars.
3. To provide the supporting features needed for real use: the ticket lifecycle with public replies and internal notes, a media library, email notifications and email-to-ticket, a dashboard, an audit trail, a REST API and signed webhooks.
4. To implement a **reporting module** that records every change to the main records and shows any record as it was at a past moment.

The algorithms are intentionally minimal and replaceable, so that they can be understood completely and later substituted with stronger techniques.

1.4 Scope and Limitation

Scope. The system covers workspace provisioning by a platform administrator; sign-in, invitations, roles and permissions within each workspace; contacts and organisations with customer tiers; teams, skills, categories, agent capacity, availability and shifts; the complete ticket lifecycle with comments, attachments and history; the four algorithms with explanations shown to agents; SLA policies with business calendars and warning and breach notifications; in-application and email notifications; email-to-ticket; a media library; a dashboard and a catalogue of reports with drill-down and point-in-time views; a REST API with client credentials and signed webhooks; light and dark themes; and deployment with Docker to any virtual machine, demonstrated on an Amazon Web Services instance.

Limitations.

- The algorithm weights and thresholds are set by administrators rather than learned from history.
- Duplicate detection compares words, so two tickets that describe the same problem in different vocabulary are not detected.
- Load testing was not carried out; the system runs on a single small virtual machine.
- A customer self-service portal, live chat, a knowledge base, custom fields and billing are outside the scope.
- The agent application is designed for desktop and laptop screens; layouts for phones are not provided.

1.5 Development Methodology

The project followed the **waterfall model** (Figure 1.1). The requirements of the system were known at the start from the problem statement and the course requirements, and the project had a fixed period of three months, from 16 June to 15 September 2026, so the work was divided into phases that were completed one after another, each ending with a document or product that was reviewed before the next phase began.

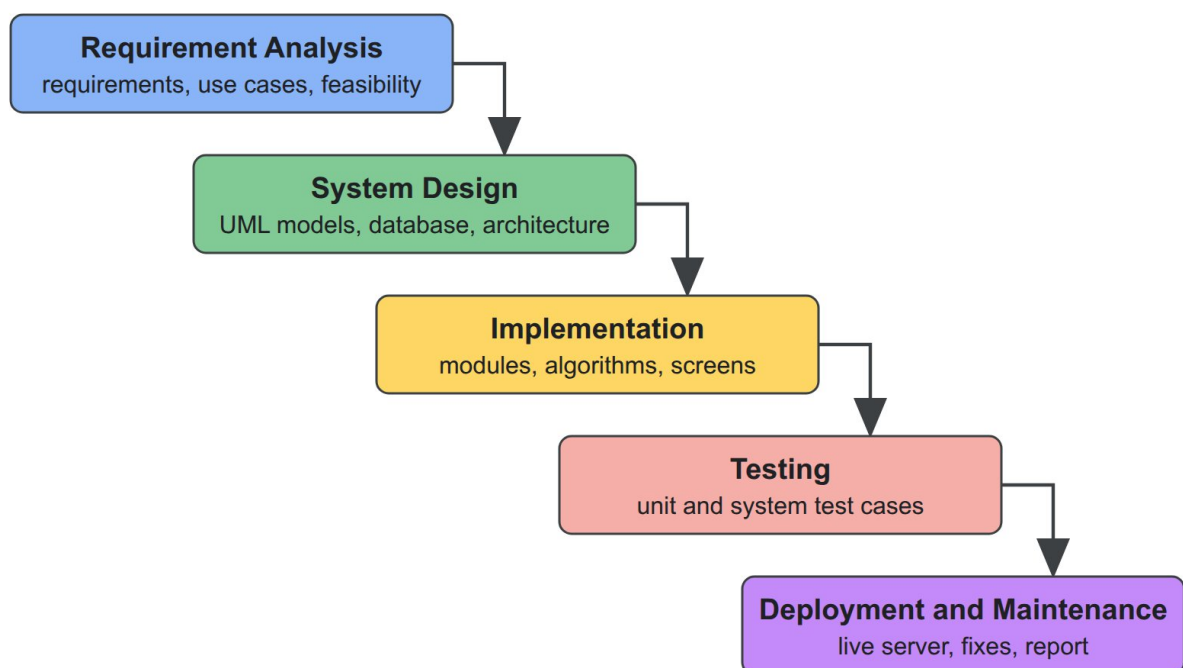


Figure 1.1: Waterfall model of the development

1. **Requirement analysis:** study of existing helpdesk systems, collection of functional and non-functional requirements, use cases and feasibility study.
2. **System design:** class, object, state, sequence and activity diagrams, database design, component and deployment design, and the design of the four algorithms.
3. **Implementation:** development of the modules, from tenancy, sign-in and the ticket model to the algorithms, SLA timers, email, reports, integrations and the user interface.
4. **Testing:** manual unit testing of each function and system testing of the complete workflows.
5. **Deployment and maintenance:** installation on a cloud virtual machine, correction of the problems found, and completion of the documentation.

The two team members worked on separate features during implementation and reviewed each other's work before it was merged. Table 1.1 lists the output of each phase.

Table 1.1: Phases of the waterfall model and their outputs

Phase	Main activities	Output
Requirement analysis	study of existing systems, requirements, use cases, feasibility	requirement specification
System design	UML models, database and architecture design, algorithm design	design documents and diagrams
Implementation	backend modules, algorithms, web application screens	working system
Testing	unit and system test cases	tested system and test report
Deployment and maintenance	cloud deployment, fixes, documentation	live system and project report

1.6 Report Organization

Chapter 2 introduces the concepts the system is built on and reviews existing helpdesk systems and related research. Chapter 3 presents the requirement and feasibility analysis, the object, dynamic and process models, the system design and the details of the algorithms. Chapter 4 describes the tools, the implementation of each module, the test cases and the analysis of the results. Chapter 5 concludes the report and recommends future work.

Chapter 2: Background Study and Literature Review

2.1 Background Study

2.1.1 Tickets, priority and service levels

In service management a ticket is a unit of work with a lifecycle. Smart Helpdesk uses six statuses (open, assigned, in progress, pending, resolved and closed) because each one changes what the system does: an open ticket without an agent is a candidate for assignment, a pending ticket waits for the customer and pauses its SLA clock, and a resolved ticket can be reopened.

Priority is commonly derived from **impact** (how much of the organisation is affected) and **urgency** (how soon the effect becomes serious); Jira Service Management, for example, maps four impact and four urgency levels to five priorities with a lookup matrix [1]. The ITIL 4 guidance notes that prioritisation also depends on the backlog and on target resolution times [2], which motivated a score that also considers how long a ticket has waited.

A **service level agreement** defines targets such as first response time and resolution time. Products differ in the details, for example whether a timer pauses while the ticket waits for the customer and whether targets count calendar or business hours. Smart Helpdesk makes these behaviours explicit settings.

2.1.2 Weighted scoring and load balancing

Simple Additive Weighting scores an alternative as the weighted sum of its normalised criteria values [3]. Operating systems use **ageing** to stop low-priority work from waiting forever by raising its priority as it waits. For distributing work, load balancers and contact centres send each request to the least-loaded server or to the least-loaded agent who has the required skill [4]. The priority and assignment algorithms of this project are built on these ideas.

2.1.3 Text similarity

Information retrieval compares documents by the words they contain. Text is split into words, lower-cased and cleared of common stop words; the **Jaccard similarity** of two

word sets is the number of shared words divided by the number of distinct words in both [5]. Richer methods weight words by how rare they are or use learned language models.

2.1.4 Multi-tenant software

A multi-tenant application serves several organisations from one deployment. Keeping all organisations in shared tables with a tenant identifier on every row is the simplest to operate but depends on every query being filtered correctly. PostgreSQL **row-level security** adds a filter enforced by the database itself, so a query that forgets the filter returns no rows instead of another organisation's data [6]. The system uses both layers.

2.2 Literature Review

2.2.1 Existing helpdesk systems

Four open-source helpdesks were studied for the capabilities this project targets (Table 2.1).

Table 2.1: Comparison of existing open-source helpdesk systems

System	Priority	Assignment	Duplicate detection	SLA
osTicket	manual levels per help topic	by department; manual claim	none	grace period after which a ticket is overdue
Zammad	three fixed levels	group rules; no workload awareness	manual merge	response and solution timers with calendars
GLPI	computed from an urgency and impact matrix	rules assign a technician or group	none	attached by rules
FreeScout	minimal	workflow rules	none	only through a paid module

Only GLPI computes priority, and it uses a fixed matrix without waiting time. None of the systems assigns tickets by measured workload and skills, and none detects duplicate tickets automatically. Smart Helpdesk combines a transparent priority score, workload-aware assignment, duplicate detection and calendar-aware SLA timers in one system that explains each decision.

2.2.2 Related research

Recent work treats ticket priority and routing as classification problems solved with machine learning on historical tickets. These models need labelled history and cannot easily explain their output, so a transparent weighted score is a reasonable starting point for a new installation. For duplicate reports, Runeson et al. compared the words of new bug reports with existing ones and found about 30 % of duplicates among the top five suggestions [7]; later work added weighted terms and learned models. For SLA timers, statecharts give a clear way to describe states that run alongside a ticket's workflow [8]. The algorithms of this project follow the simplest of these approaches and are built so that stronger methods can replace them later.

Chapter 3: System Analysis and Design

3.1 System Analysis

3.1.1 Requirement Analysis

i. Functional requirements

The actors of the system are the Platform Super Admin, who manages workspaces; the Tenant Admin, who configures a workspace; the Support Manager, who oversees queues and reports; the Support Agent, who works tickets; external systems, which use the API and receive webhooks; the Contact, who raises requests by email; and the scheduler, which drives time-based processing. Figures 3.1 and 3.2 show the use cases.

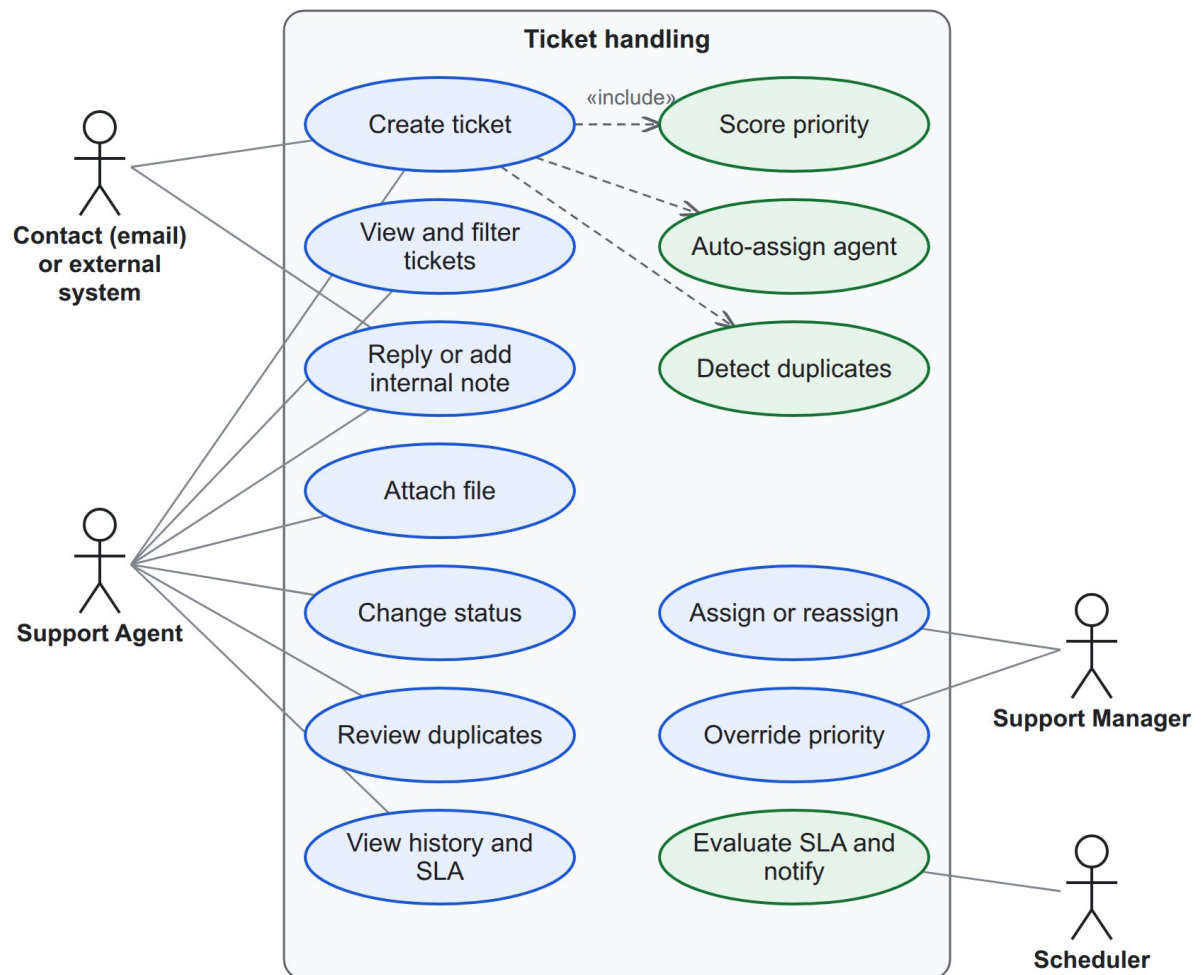


Figure 3.1: Use case diagram of ticket handling and automation

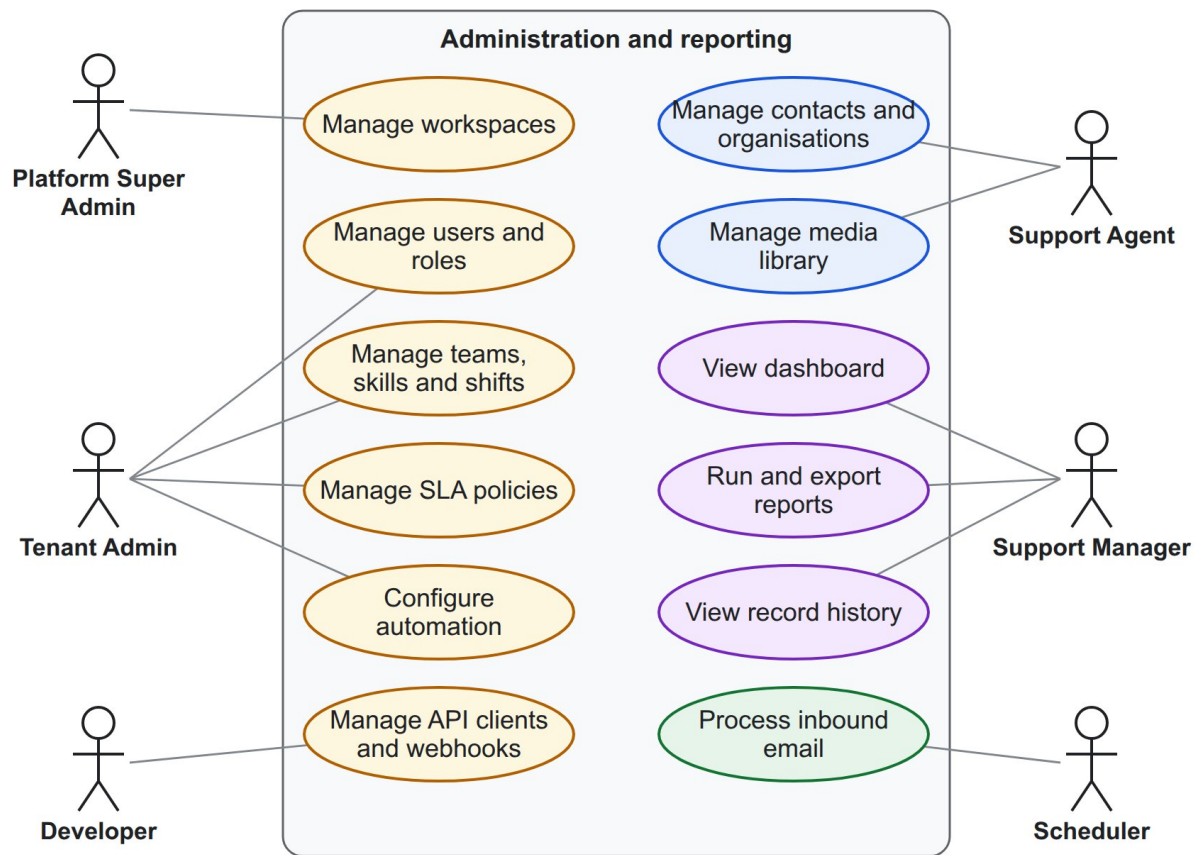


Figure 3.2: Use case diagram of administration, contacts and reporting

Table 3.1: Functional requirements by module

Module	Requirement
Tenancy	Create, suspend and reactivate workspaces; take the workspace of every request from the signed-in session or API client; seed default roles, categories, SLA policy and settings
Identity	Sign in with email and password; invite users; assign roles made of permissions; check a permission on every action
Contacts and agents	Manage contacts and organisations with customer tier; teams, skills, categories, agent capacity, availability and shifts
Tickets	Create tickets; move them through the lifecycle; add public replies and internal notes; attach files; keep history; list, filter and search
Automation	Score priority; assign agents; suggest duplicates; explain every automatic decision
SLA	Define policies and business calendars; run first-response and resolution timers; pause while pending; warn before and record breaches
Media and email	Store files in a workspace library; send notification emails; turn customer replies into comments and new emails into tickets

Module	Requirement
Integrations	REST API, client credentials, API documentation, signed webhooks with retries
Reporting	Record every change to the main records; report catalogue with filters and drill-down; record history and point-in-time view; CSV and XLSX export

Table 3.2: Description of use case "Create ticket"

Field	Description
Actors	Support Agent; external system through the API; Contact through email
Precondition	The actor is signed in to the workspace (or the email is routed to it)
Main flow	1. Actor enters title, description, contact, category, impact, urgency and attachments. 2. System shows likely duplicates. 3. Actor confirms. 4. System numbers the ticket, computes its priority, starts SLA timers and assigns an agent. 5. System records history and sends notifications.
Alternative flows	2a. Actor marks the ticket as a duplicate; it is closed and linked to the original. 4a. No agent is eligible; the ticket stays unassigned and managers are notified.
Postcondition	The ticket exists with number, priority, explanation, SLA timers and assignment

Table 3.3: Description of use case "Assign ticket"

Field	Description
Actors	System (automatic), Support Manager (manual)
Main flow	1. System keeps the agents who are available, on shift, below capacity, in the team and hold the required skills. 2. System orders them by open tickets per unit of capacity. 3. System assigns the first agent and stores the ordered list. 4. Agent is notified.
Alternative flow	1a. A manager reviews the ranking and chooses a different agent with a reason.

ii. Non-functional requirements

Table 3.4: Non-functional requirements

Quality	Requirement
Security	No access to another workspace through any screen or API; permission check on every action; sign-in throttling; signed webhooks and emails;

Quality	Requirement
	private file storage
Performance	Ticket list and ticket creation respond within about one second for a workspace with many thousands of tickets
Reliability	Background work is queued and retried; daily backups with a tested restore
Usability	Light and dark themes; keyboard operation; readable contrast; loading, empty and error states on every screen
Portability	Runs on any Linux virtual machine with Docker

3.1.2 Feasibility Analysis

i. Technical feasibility. All components are mature open-source software: Laravel on PHP, React, PostgreSQL, Valkey, object storage and a mail server. They run together with Docker Compose on one virtual machine; the live deployment uses an Amazon Web Services instance with two virtual CPUs and 2 GB of memory.

ii. Operational feasibility. The system follows the workflow support teams already use and explains every automatic decision, so agents can trust or override it. Administrators adjust the algorithms through forms rather than code.

iii. Economic feasibility. No software licence is required. A virtual machine of the required size costs roughly US\$8 to 35 per month, and an organisation's own server can be used instead. Development used free tools.

iv. Schedule feasibility. The project was planned over three months, from 16 June to 15 September 2026, following the phases of the waterfall model (Section 1.5), each phase starting when the previous one was complete, with documentation written throughout (Table 3.5 and Figure 3.3).

Table 3.5: Project schedule

S.N.	Phase	Start	End	Duration
1	Requirement analysis	16 Jun 2026	29 Jun 2026	2 weeks
2	System design	30 Jun 2026	13 Jul 2026	2 weeks
3	Implementation	14 Jul 2026	24 Aug 2026	6 weeks
4	Testing	25 Aug 2026	7 Sep 2026	2 weeks
5	Deployment and maintenance	8 Sep 2026	15 Sep 2026	8 days
6	Documentation and report	16 Jun 2026	15 Sep 2026	3 months

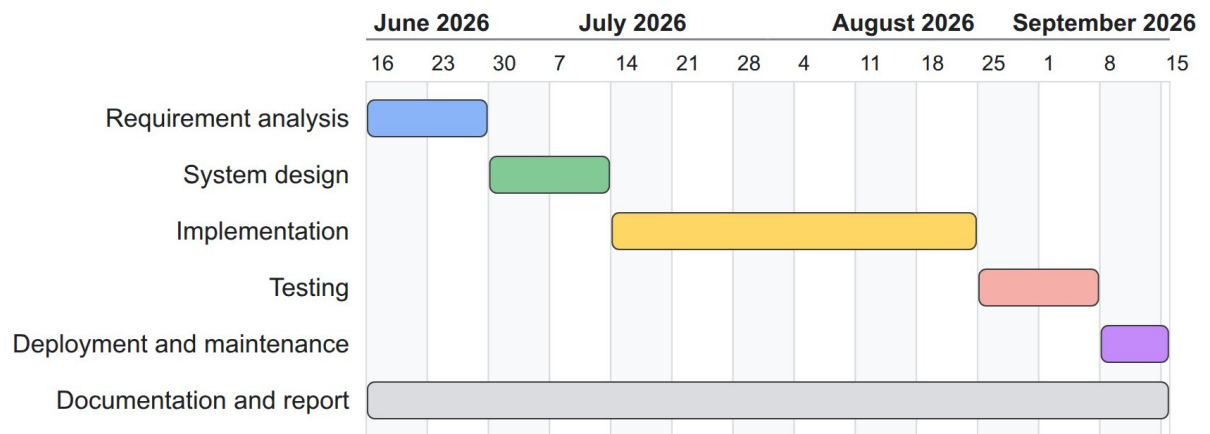


Figure 3.3: Gantt chart of the project schedule

3.1.3 Object Modelling using Class and Object Diagrams

The domain model (Figure 3.4) is centred on the **Ticket**. A ticket belongs to one workspace, is requested by one contact (who may belong to an organisation with a customer tier), has one category, and may be assigned to a team and an agent. A ticket has comments, history events, assignments, two SLA timers, duplicate suggestions and attached files. A category requires skills; an agent holds skills, belongs to teams and has shifts. An SLA policy has targets per priority level and uses a business calendar with holidays. For reporting, a change record stores every change of a main record.

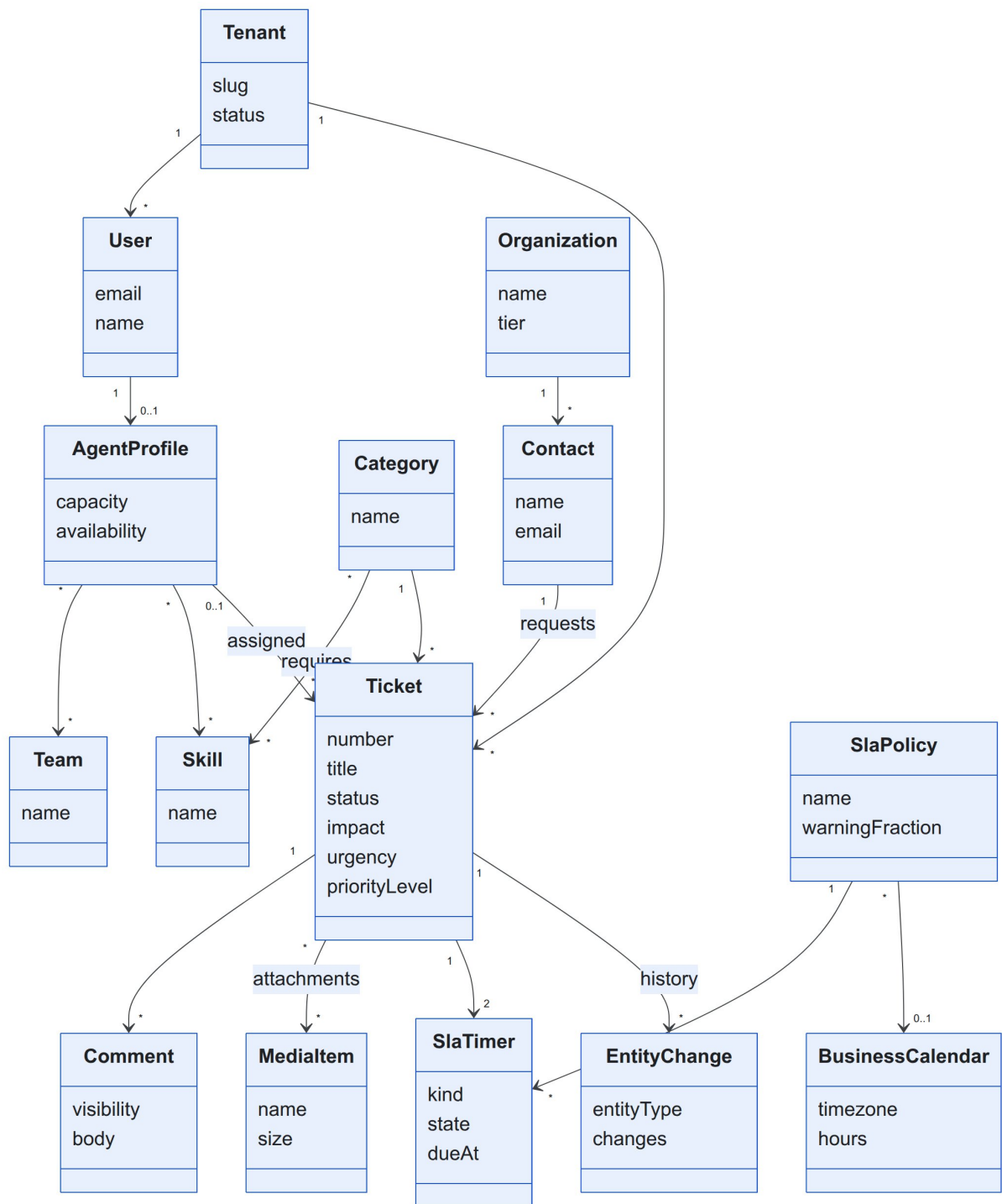


Figure 3.4: Class diagram of the domain model

Figure 3.5 shows an object diagram of a seeded ticket: ticket #1042 requested by a contact of a premium organisation, in the Billing category, assigned to agent Asha of the Finance Support team, with an eight-hour resolution timer.

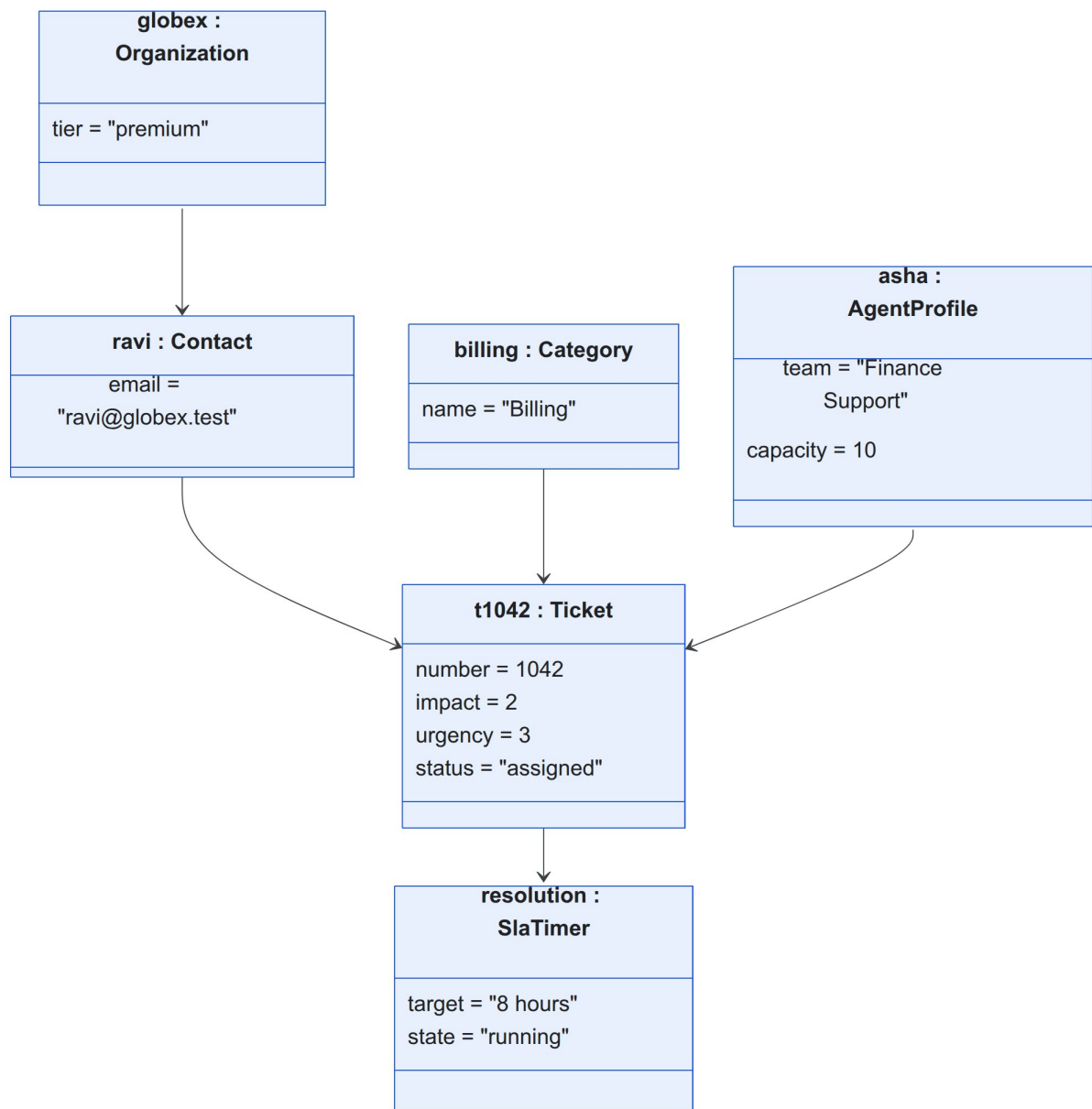


Figure 3.5: Object diagram of a seeded ticket

3.1.4 Dynamic Modelling using State and Sequence Diagrams

The ticket lifecycle (Figure 3.6) has six states. A new ticket is open; assignment moves it to assigned; work moves it to in progress; waiting for the customer moves it to pending; a resolved ticket is closed manually or automatically and can be reopened. Any other transition is rejected.

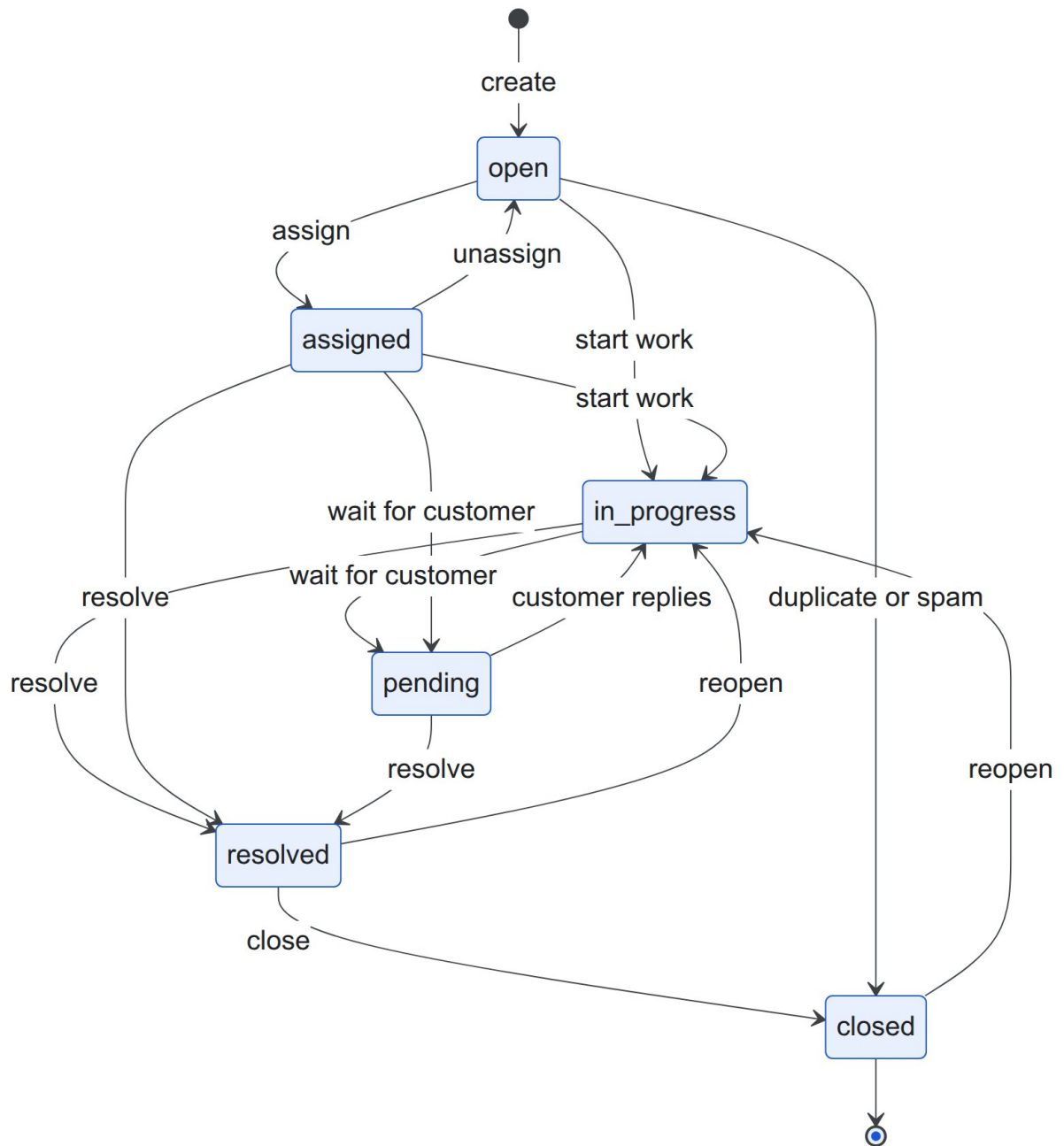


Figure 3.6: State diagram of the ticket lifecycle

Each ticket also has SLA timers that run alongside the lifecycle (Figure 3.7). A timer is running, paused, in warning, breached, met or cancelled.

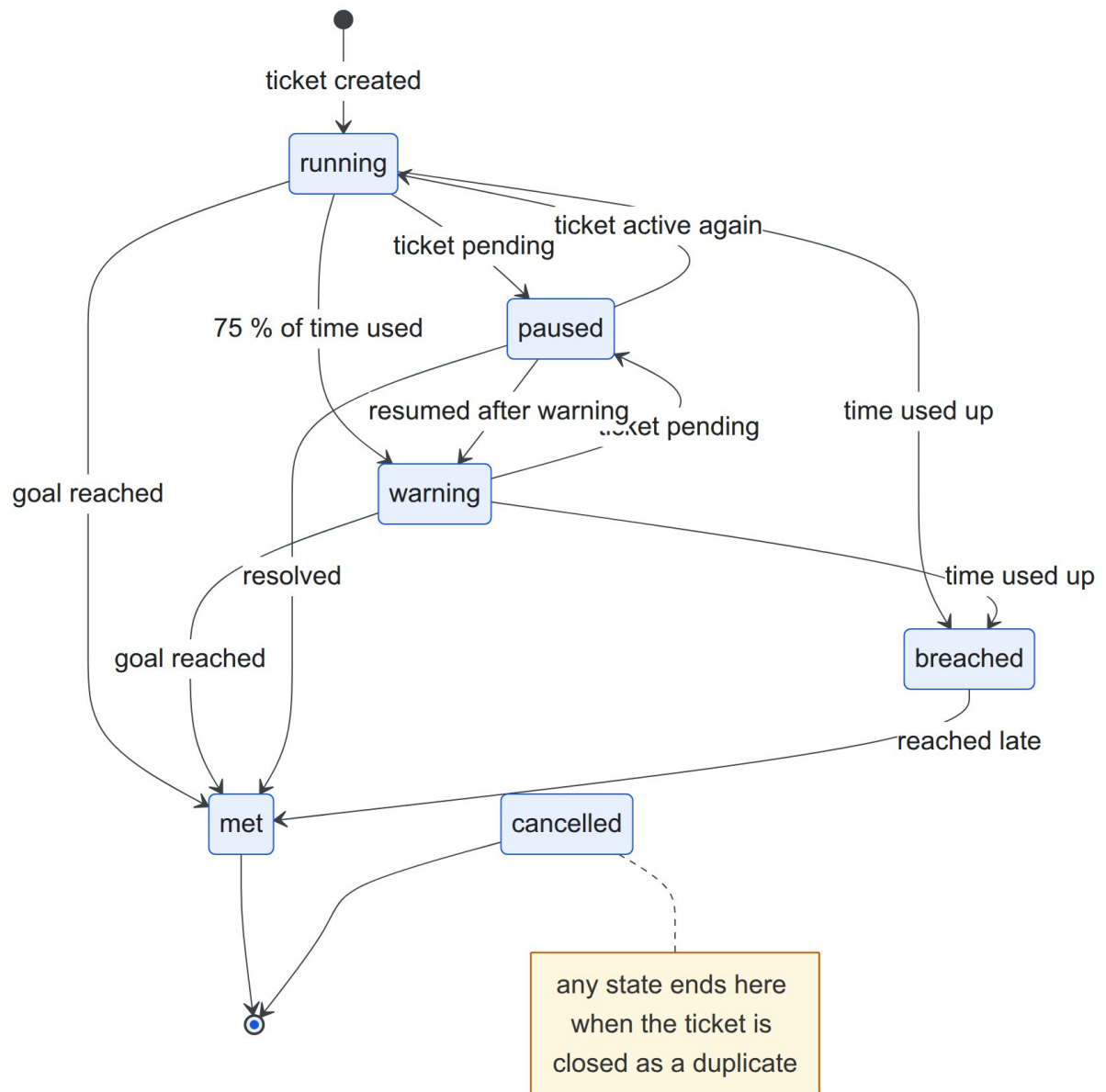


Figure 3.7: State diagram of an SLA timer

Figure 3.8 shows the creation of a ticket, in which priority, SLA, duplicate and assignment decisions are made in one database transaction. Figure 3.9 shows the SLA check that runs every minute, and Figure 3.10 the processing of an inbound email.

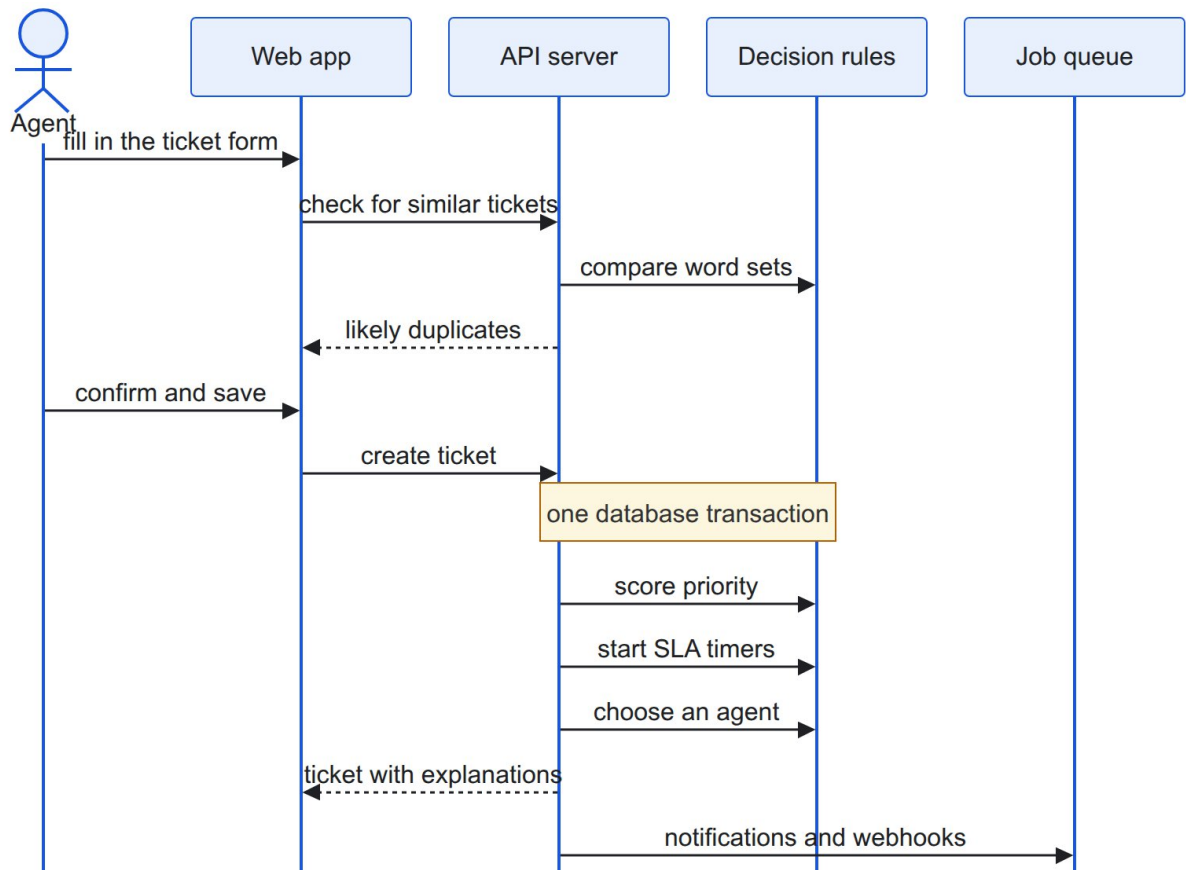


Figure 3.8: Sequence diagram of ticket creation

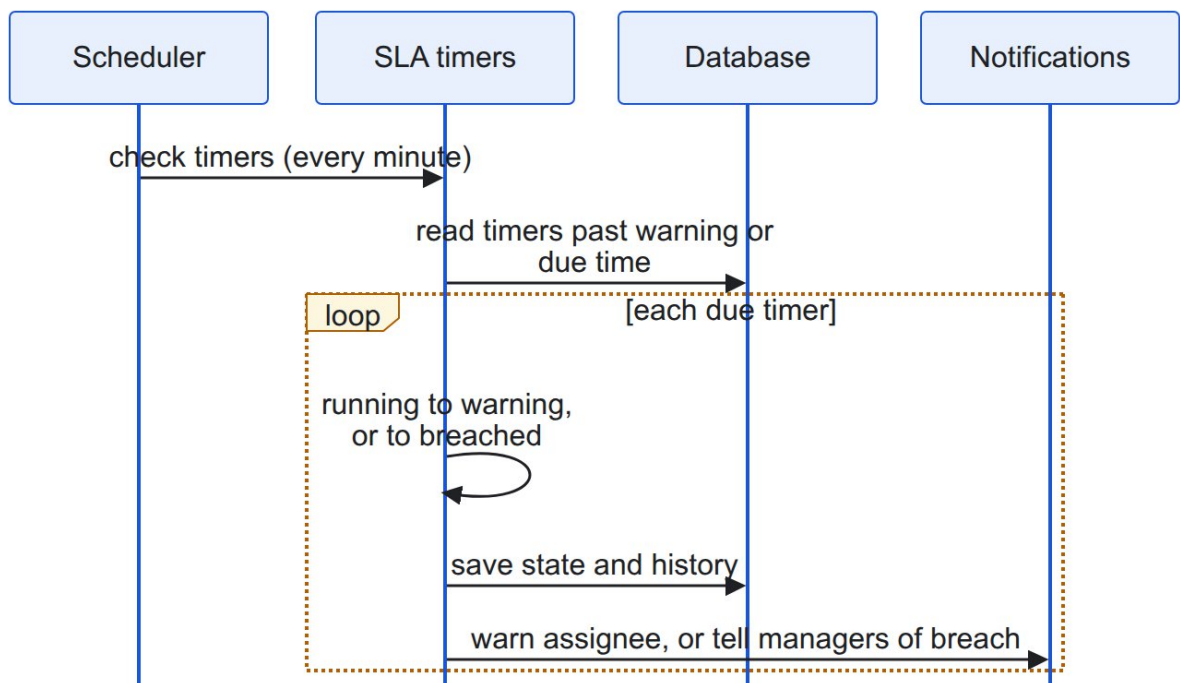


Figure 3.9: Sequence diagram of the SLA check

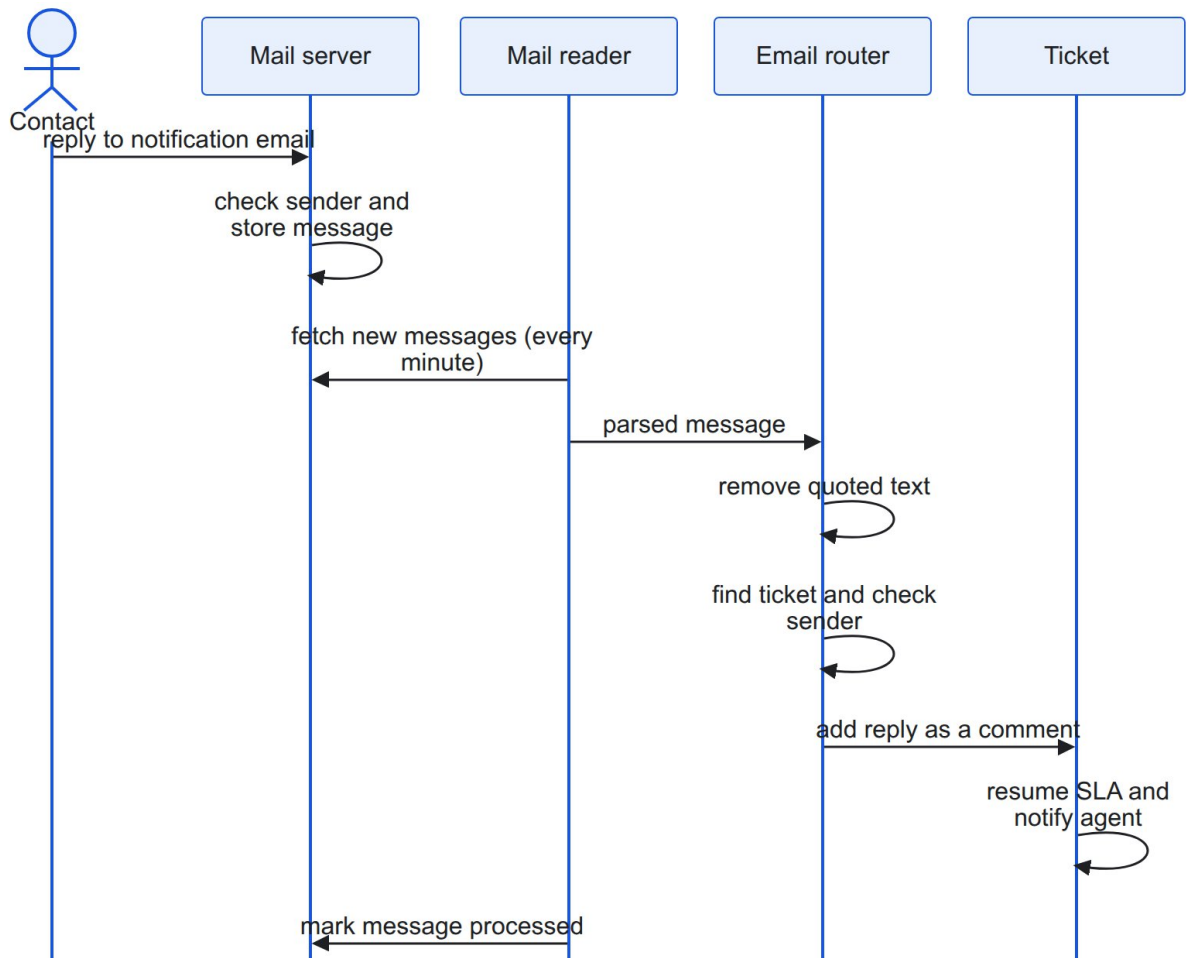


Figure 3.10: Sequence diagram of inbound email processing

3.1.5 Process Modelling using Activity Diagrams

Figure 3.11 shows agent assignment, including the case where no agent is eligible; Figure 3.12 shows duplicate detection; and Figure 3.13 shows how an inbound email becomes a comment, a new ticket or a rejection.

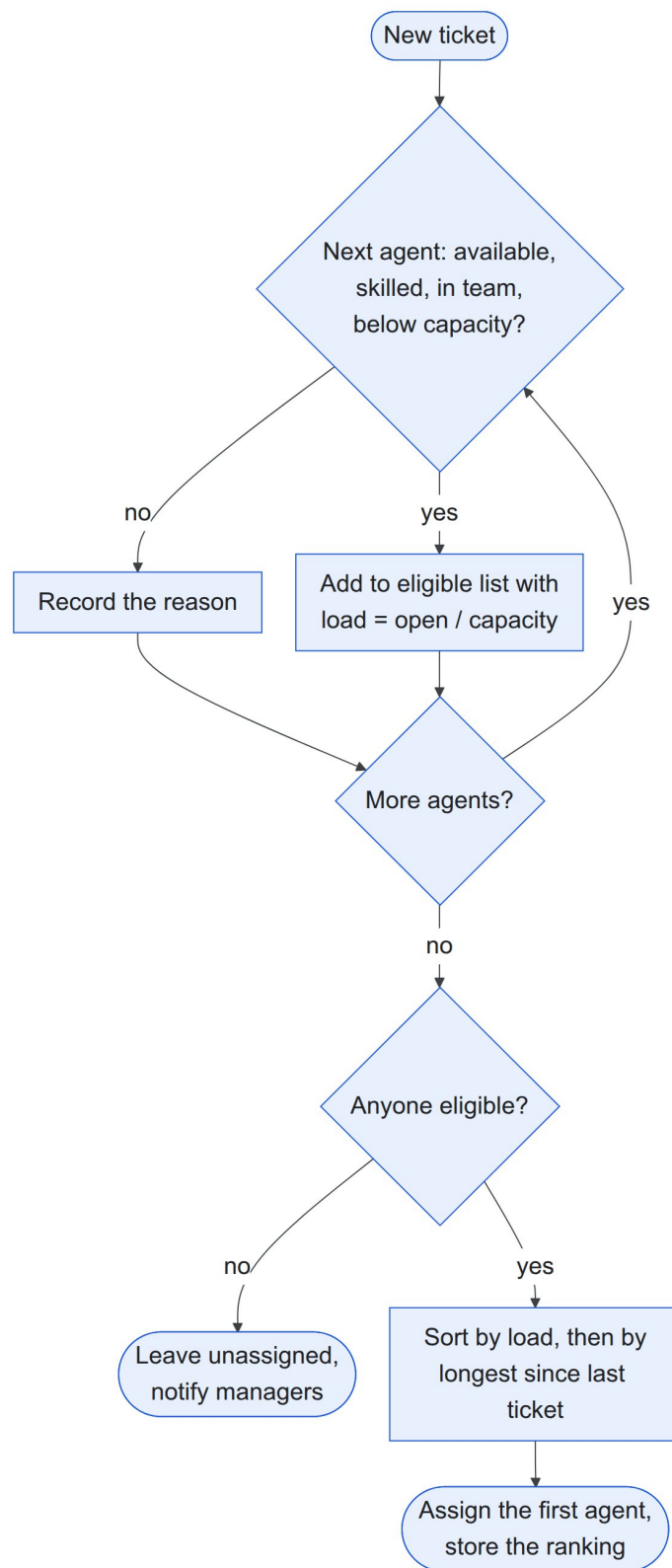


Figure 3.11: Activity diagram of agent assignment

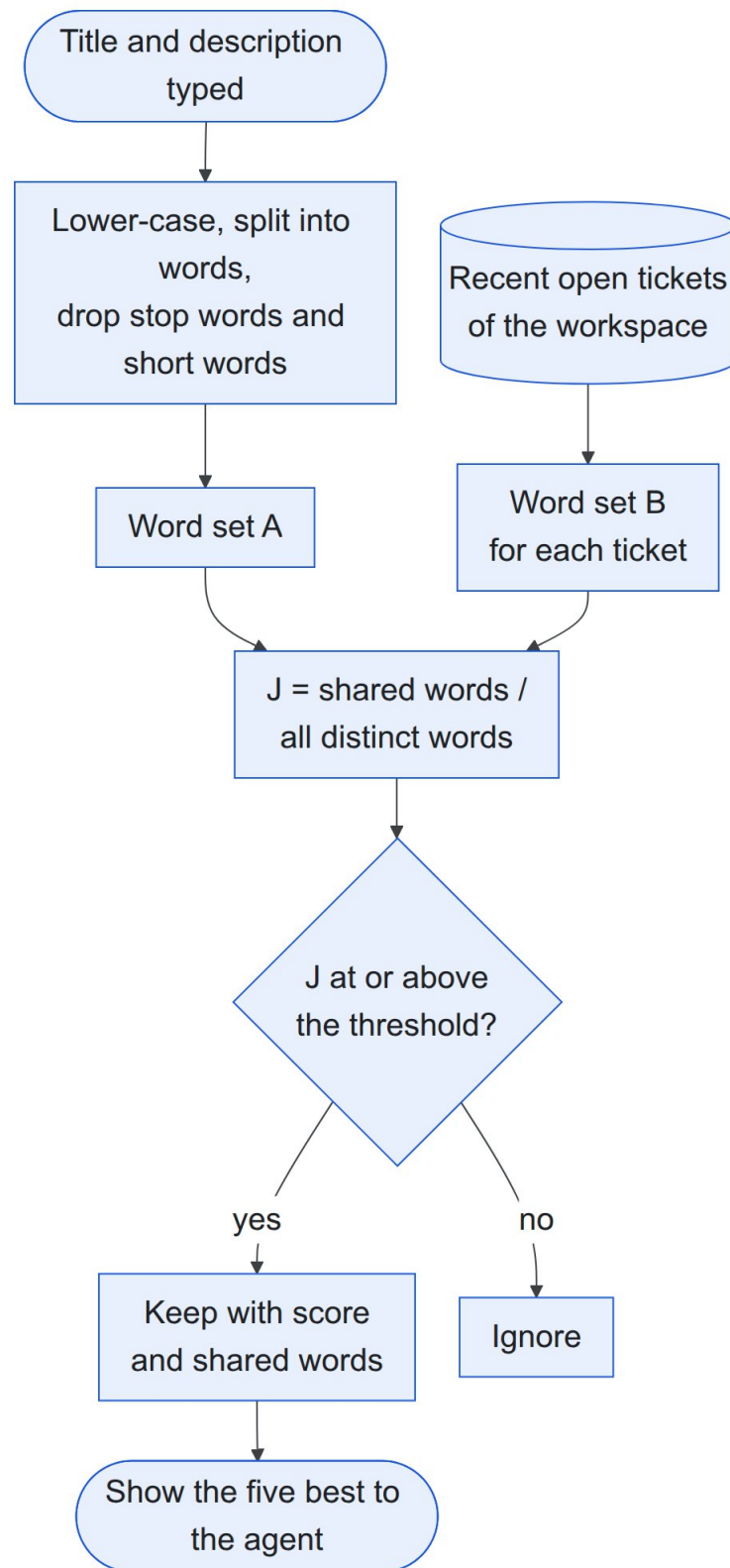


Figure 3.12: Activity diagram of duplicate detection

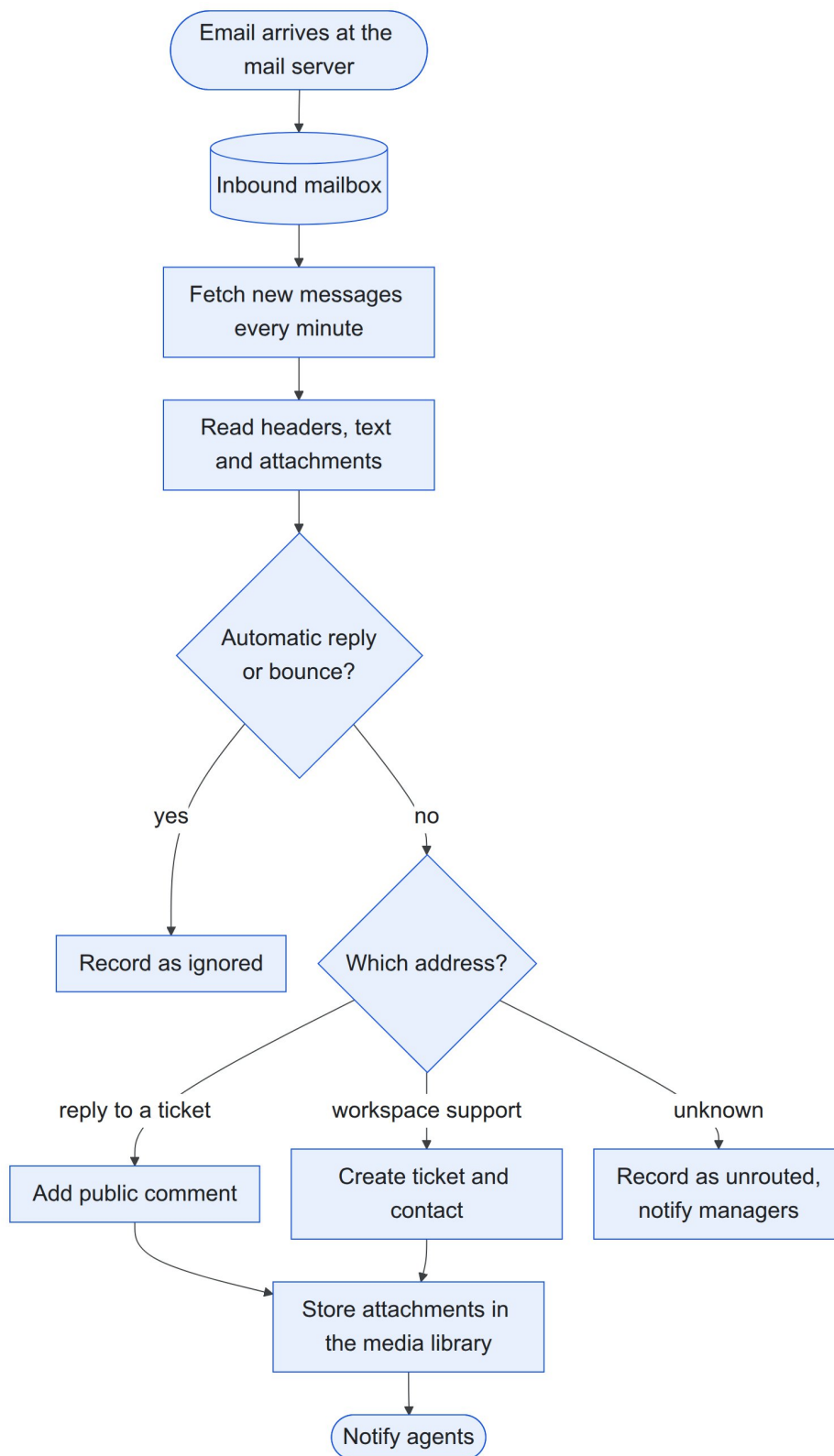


Figure 3.13: Activity diagram of inbound email routing

3.2 System Design

3.2.1 Refinement of Class, Object, State, Sequence and Activity Diagrams

The analysis classes were refined into modules. Each algorithm became an interface with a simple baseline implementation that receives plain input values and returns a result together with its explanation (Figure 3.14), so that a stronger algorithm can replace it without changing the rest of the system. Each write operation, such as creating, assigning or resolving a ticket, became a single step that runs inside one database transaction and announces what happened once it is saved. The ticket state diagram became a transition table, and the SLA state diagram a table of allowed timer transitions.

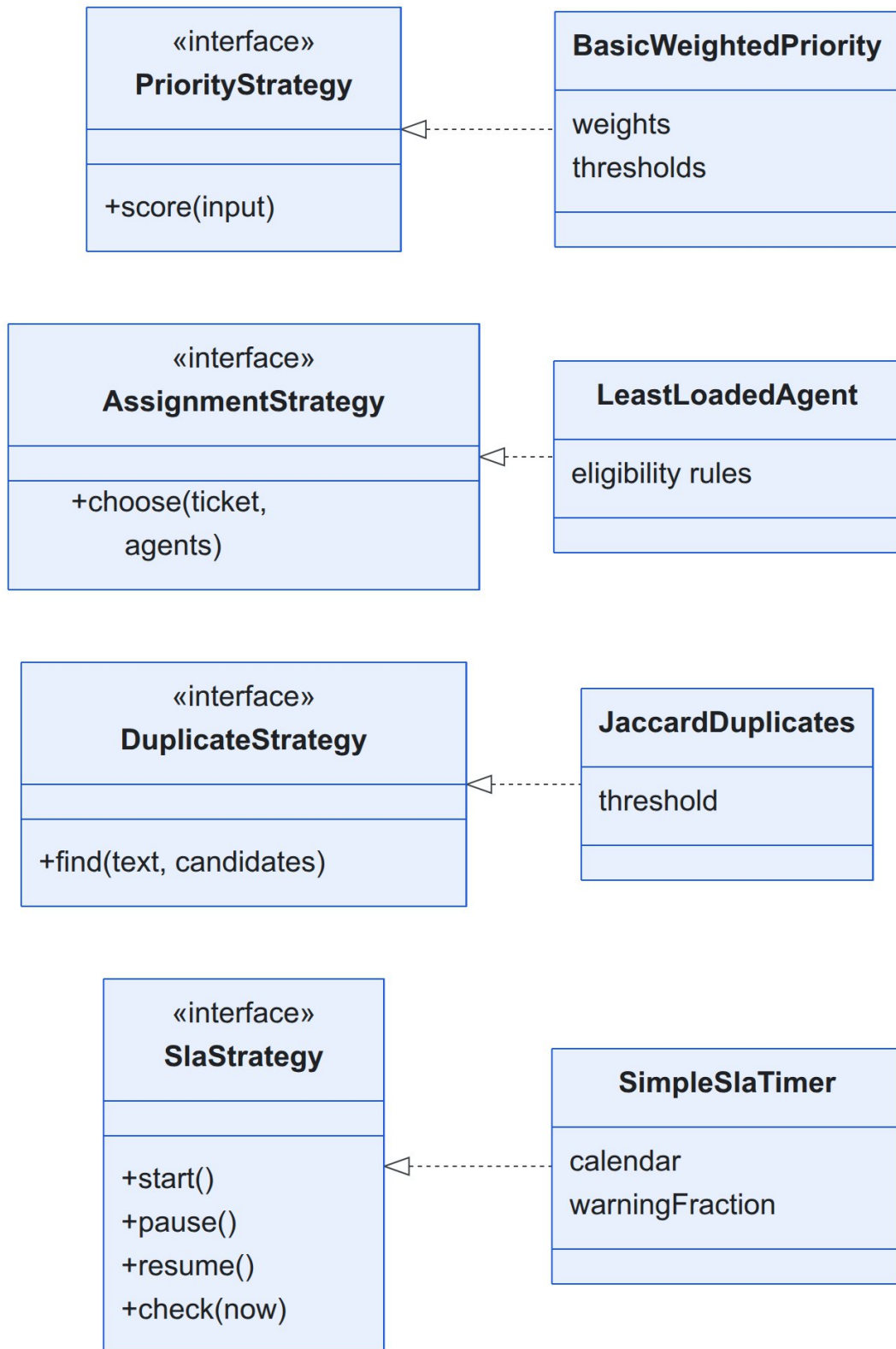


Figure 3.14: Algorithm interfaces and their baseline implementations

The database design (Figure 3.15) follows the refined classes. Every workspace-owned table carries the workspace identifier and a row-level security policy; ticket numbers are allocated per workspace; and text indexes support search and duplicate candidate retrieval.

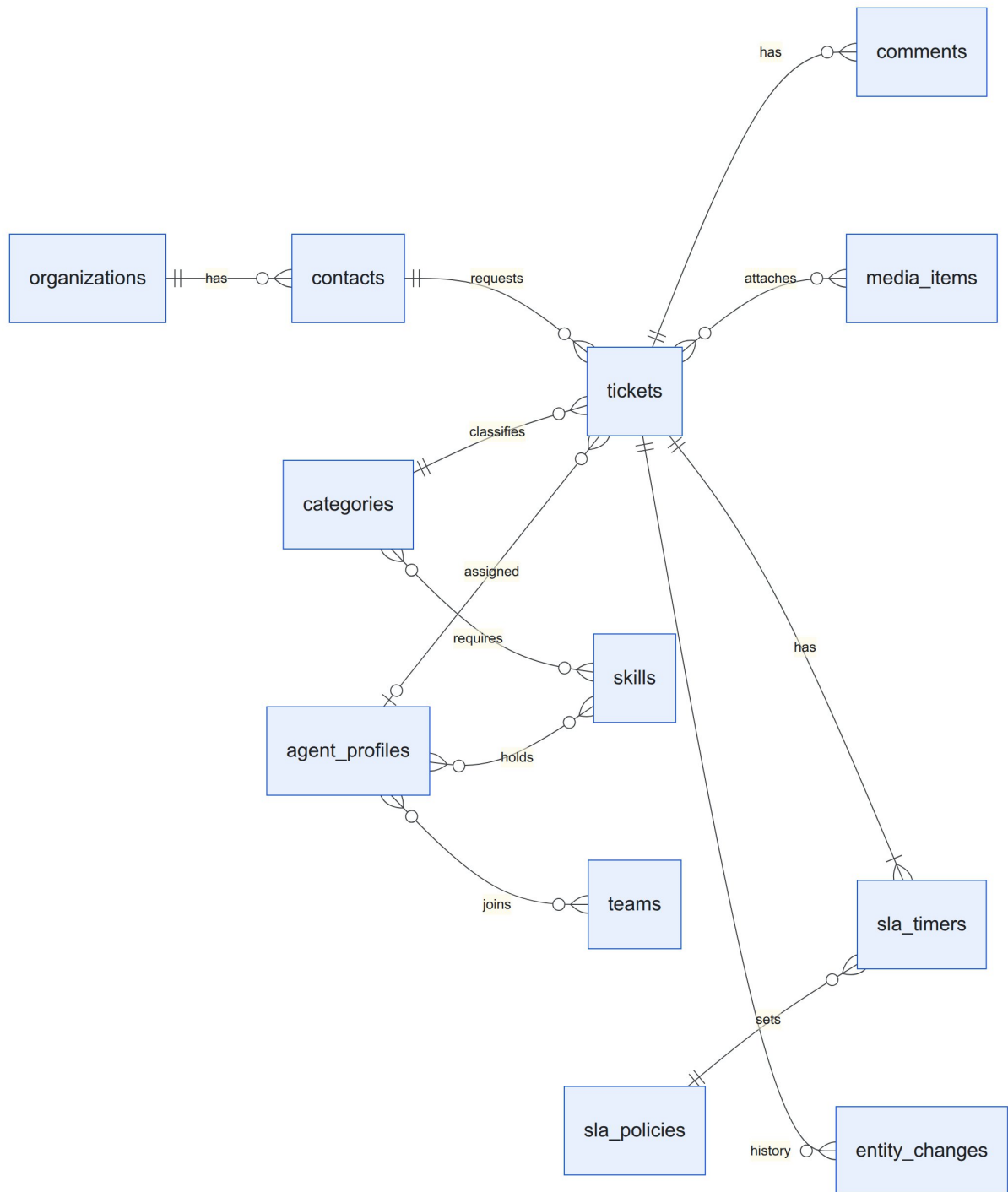


Figure 3.15: Entity relationship diagram

3.2.2 Component Diagrams

The backend is divided into modules (Figure 3.16): platform, tenancy, identity, contacts, agents, tickets, SLA, automation, media, mail, notifications, integrations, reporting and audit. Dependencies point in one direction, and notification, integration, reporting and audit components only react to events. The frontend is a single-page application divided into features on top of a shared design system.

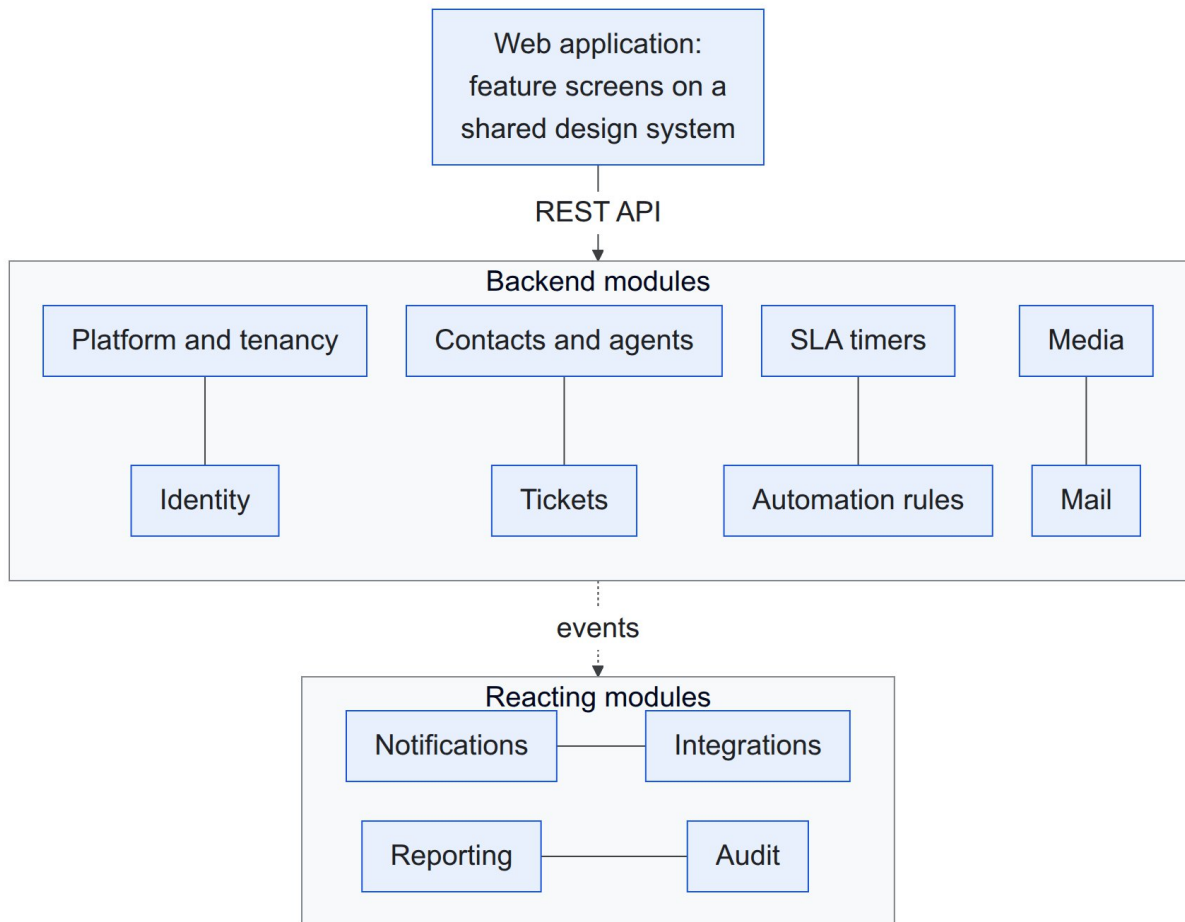


Figure 3.16: Component diagram

3.2.3 Deployment Diagrams

Figure 3.17 shows the deployment on one virtual machine. A reverse proxy terminates HTTPS for the application, API, administration, documentation, file and mail hosts and serves the web application; the PHP application, queue workers and scheduler run from one container image; PostgreSQL, Valkey, object storage and the mail server run as separate containers on an internal network. The same arrangement runs on a developer computer, an organisation's own server or any cloud virtual machine.

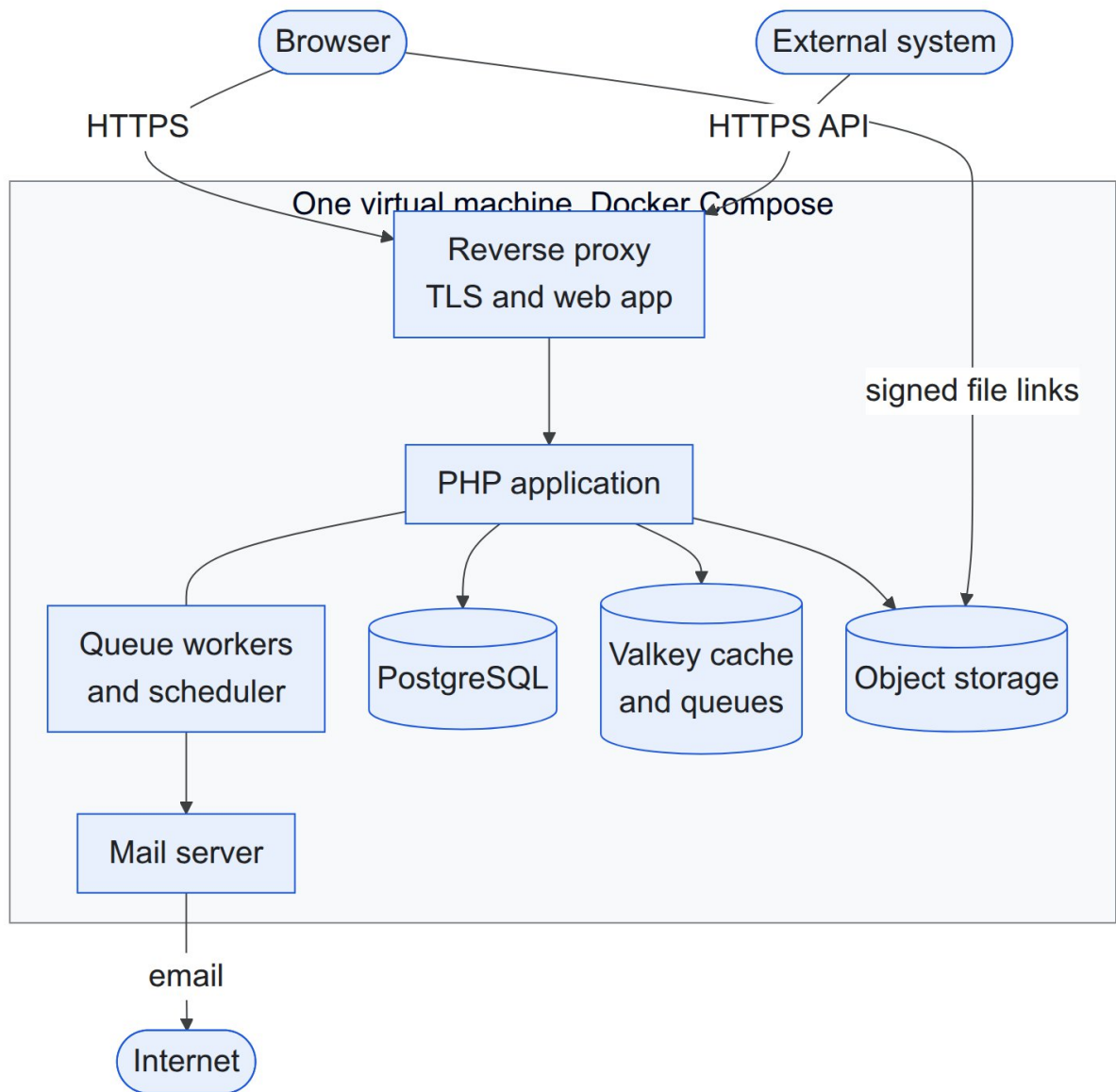


Figure 3.17: Deployment diagram

3.3 Algorithm Details

The four decision algorithms are deliberately simple baselines. Each is implemented behind an interface (strategy) so that it can be replaced by a stronger technique after this project without changing the rest of the system, and each decision stores the name and version of the strategy that made it.

3.3.1 Priority scoring: basic weighted priority

The priority score is a weighted sum of four values scaled to the range 0–1, a form of simple additive weighting [3]. Impact and urgency are the inputs commonly used by service-

management tools [1]; customer tier reflects the importance of the requester; waiting time is an ageing term that raises tickets that have waited long.

Table 3.6: Inputs of the priority score

Input	Values	Scaled value	Weight
Impact	1 (one user) to 4 (whole organisation)	$(\text{impact} - 1) \div 3$	0.40
Urgency	1 (low) to 4 (immediate)	$(\text{urgency} - 1) \div 3$	0.35
Customer tier	standard, premium, enterprise	0, 0.5, 1	0.15
Waiting time	hours since creation	$\min(1, \text{hours} \div 72)$	0.10

```
score = 100 × (0.40 × impact' + 0.35 × urgency' + 0.15 × tier' + 0.10 ×
age')
level = P1 if score ≥ 75, P2 if score ≥ 50, P3 if score ≥ 25, otherwise P4
```

```
function score(ticket, now):
    total ← 0
    for each (name, weight, scaled value) of the four inputs:
        part ← 100 × weight × value
        total ← total + part
        record (name, value, weight, part) in the explanation
    return total, level(total), explanation
```

The score is computed on creation, when an input changes and hourly for open tickets; a manager's manual priority always takes precedence. The cost is constant per ticket. For example, a ticket with impact 2, urgency 3 and a premium customer scores $13.3 + 23.3 + 7.5 + 3.3 = 47.5$ (P3) after 24 hours and 54.2 (P2) after 72 hours, which shows the ageing effect.

3.3.2 Agent assignment: least-loaded eligible agent

The rule combines the least-loaded choice of load balancers with the skill filter used in contact centres [4]. An agent is eligible when available (and on shift if shifts are enforced), holding all skills the category requires, in the ticket's team if one is set, and below capacity. Among eligible agents the one with the smallest ratio of open tickets to capacity is chosen; ties go to the agent assigned least recently, which behaves like round robin.

```
function choose(ticket, agents):
    eligible ← agents that pass every eligibility rule (others recorded with
the failed rule)
    if eligible is empty: leave the ticket unassigned and notify managers
    sort eligible by (open ÷ capacity, last assignment time, id)
    return the first agent and the sorted list as explanation
```

The cost is $O(A \log A)$ for A agents. For example, if Asha and Chen both have 3 of 10 tickets and Bikram has 2 of 5, Bikram's load (0.40) is higher than theirs (0.30); Chen, assigned earlier than Asha, receives the ticket.

3.3.3 Duplicate detection: Jaccard word overlap

Following the first duplicate-report detectors, which compared the words of a new report with existing reports [7], each ticket is reduced to a set of words: the title and description are lower-cased, split on characters other than letters, digits and hyphens, and stripped of a fixed list of 117 stop words and of words shorter than three characters; hyphens are kept so that error codes such as `err-401` survive, and any Unicode letter counts, so Devanagari words stay whole. Two sets are compared with the Jaccard similarity [5]:

$$J(A, B) = |A \cap B| \div |A \cup B|$$

```
function find(ticket, candidates):           // same tenant, last 30
days, not closed, at most 50
    A ← words(ticket)
    for c in candidates:
        B ← words(c)
        if |A ∩ B| ÷ |A ∪ B| ≥ 0.35: keep (c, score, A ∩ B)
    return the five highest scores
```

For "Cannot login after password reset" against "Login fails after resetting password", the sets share five of ten distinct words ($J = 0.50$) and the older ticket is suggested; an unrelated invoice ticket scores 0.15 and is ignored. The comparison costs $O(K \times W)$ for K candidates of about W words. The method is transparent but misses paraphrases and word forms, which a stronger replacement would address.

3.3.4 SLA evaluation: simple SLA timer

Each ticket has a first-response timer and a resolution timer, each a small state machine [8] with the states running, paused, warning, breached, met and cancelled. Deadlines are computed through a calendar that counts either all hours or only the organisation's working hours.

```
start:          due ← cal.add(now, target); warn ← cal.add(now, f ×
target) // f: the policy's warning fraction, default 0.75
enter pending:  paused_at ← now
leave pending:  d ← cal.elapsed(paused_at, now); due ← cal.add(due, d);
warn ← cal.add(warn, d)
priority change: due ← cal.add(started_at, new target + paused time)
goal reached:   state ← met (first public reply, or ticket resolved)
every minute:   running and now ≥ warn → warning, notify assignee (once)
                not met and now ≥ due → breached, notify assignee and
managers (once)
```

For a P2 ticket with an eight-hour resolution target created at 09:00 and waiting for the customer from 10:00 to 12:00, the warning moves from 15:00 to 17:00 and the deadline from 17:00 to 19:00. With office hours of 10:00–17:00, an eight-hour target starting on Thursday at 15:00 ends on Friday at 16:00. The minute check reads only due timers through an index and records when each notification was sent, so running it twice has no additional effect.

3.3.5 History reconstruction and time analytics

A database trigger records each insert, update and delete on reportable tables as a change row containing only the attributes that changed, with their old and new values, the actor and the time. The state of a record at a past instant t is rebuilt by starting from the current row and undoing, newest first, every change made after t :

```
function asOf(entity, t):
  changes ← changes of entity with occurred_at > t, newest first
  state ← current row (or the last values stored in its delete change)
  for c in changes:
    if c is the creation: return "did not exist at t"
    for (attribute, old, new) in c: state[attribute] ← old
  return state
```

For analysis over time, each ticket's ordered events are swept into non-overlapping intervals of constant status, assignee, team and priority:

```
function intervals(events, calendar, now):
  state ← initial state; start ← creation time; result ← []
  for e in events ordered by time:
    next ← apply(state, e)
    if next ≠ state:
      result.append(state, start, e.time, calendar.elapsed(start,
e.time))
      state, start ← next, e.time
  result.append(state, start, open, calendar.elapsed(start, now))
  return result
```

Time in a status is the sum of the matching intervals; the backlog at an instant is the number of unresolved intervals containing it; an agent's workload over time is the priority-weighted count of that agent's intervals. Reconstruction costs $O(k)$ for k later changes and interval building $O(e)$ for e events of a ticket. Daily snapshots are computed from the intervals after midnight in the organisation's time zone, and every derived table can be rebuilt from the recorded history and checked for drift.

Chapter 4: Implementation and Testing

4.1 Implementation

4.1.1 Tools Used

Table 4.1 lists the tools and technologies used to build and test the system.

Table 4.1: Tools and technologies

Category	Tool	Purpose
Backend	PHP 8.5, Laravel 13	API, background jobs, scheduling, mail, validation
Frontend	React 19, TypeScript, Vite	single-page web application
Frontend libraries	TanStack Router, Query, Table and Form; Zod	routing, server data, tables, forms, validation
User interface	Tailwind CSS, shadcn/ui, Lucide icons, Recharts	design system, components, icons, charts
Database	PostgreSQL 18	relational data, text search, row-level security
Cache and queues	Valkey, Laravel Horizon	cache, background job queues and their monitoring
Files and email	RustFS object storage, Stalwart mail server	file storage, sending and receiving email
API documentation	Scramble	interactive API reference
Infrastructure	Docker Compose, Caddy, Ansible, OpenTofu	containers, HTTPS proxy, server setup, cloud provisioning
Diagrams	Mermaid	use case, class, object, state, sequence, activity, component and deployment diagrams
Testing	Google Chrome, Mozilla Firefox	manual testing of every screen and workflow
Editor and version control	Visual Studio Code, Git, GitHub	development and source control

4.1.2 Implementation Details of Modules

The backend is divided into modules, each with its own routes, data models, business rules and database tables, and a module may depend only on the modules below it. Every change of data runs as one step inside a database transaction, so a failure leaves nothing half saved.

The four algorithms are written as plain functions behind interfaces, so the rest of the system never depends on a particular algorithm.

Tenancy and platform. After sign-in, the workspace of every request is taken from the user's session or from the API client that made the request, never from the web address. Requests from users of another workspace and requests to a suspended workspace are refused. The database then limits every query to the current workspace through row-level security. A new workspace is created with default roles, categories, an SLA policy, a calendar and settings. The platform administrators have their own sign-in, separate from the workspaces.

Identity. Users sign in with email and password, accept invitations and reset forgotten passwords by email. Roles are sets of permissions within a workspace, such as "assign tickets" or "manage SLA policies", and every action checks the permission it needs.

Tickets. Creating a ticket gives it the next number of its workspace and, in the same transaction, computes its priority, starts its SLA timers, looks for duplicates and assigns an agent. Status changes are allowed only along the lifecycle of Figure 3.6. Replies, internal notes and attachments are stored with the ticket, and the ticket list supports filtering, sorting, text search and paging.

Automation. The priority, assignment and duplicate algorithms each return their decision together with an explanation, which is stored and shown to agents in words. A manager may override a priority or an assignment with a recorded reason.

SLA. Each ticket has a first-response timer and a resolution timer that start, pause, resume, finish and recompute as the ticket changes. Deadlines are counted on a business calendar with working hours and holidays, and a scheduled task checks due timers every minute and sends warning and breach notifications.

Media and mail. Files are uploaded directly to object storage through short-lived signed links, checked for size and type, and given thumbnails. Notification emails carry a reply address that identifies the ticket; incoming emails are read from the mail server every minute, cleaned of quoted text and added as a comment or turned into a new ticket.

Reporting. A database trigger records every change to the main tables with the old and new values. From these records the system can show any record as it was at a past moment, build the periods a ticket spent in each state, keep daily summaries, and run a catalogue of reports that can be filtered, drilled into and exported as CSV or XLSX files.

Integrations. External systems obtain an access token with their client identifier and secret and can use only the permissions granted to them. Webhook messages are signed with a shared secret so that receivers can verify them, and failed deliveries are retried.

Frontend. The web application is organised by feature on a shared design system with light and dark themes. Lists keep their filters in the address bar, forms warn about unsaved changes, and every automatic decision is shown in words with its explanation. Figures 4.1 to 4.3 show the main screens; more screens are in the appendix.

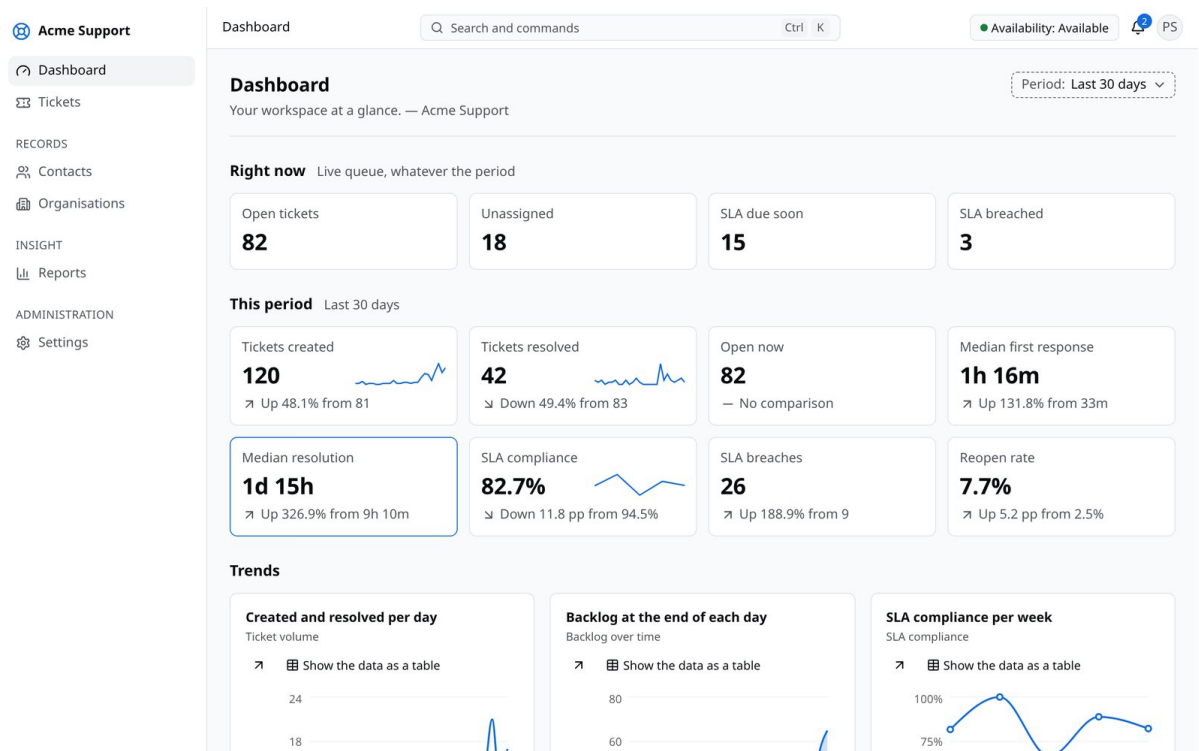


Figure 4.1: Management dashboard

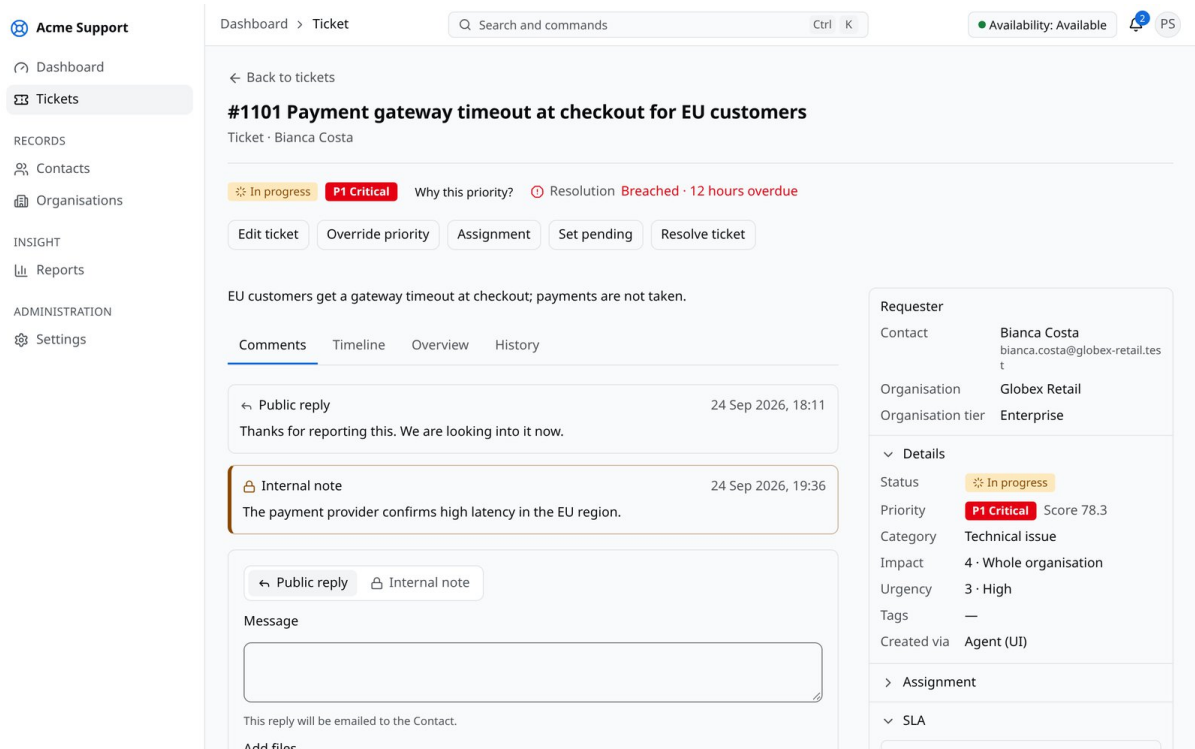


Figure 4.2: Ticket screen with conversation, SLA timers and details

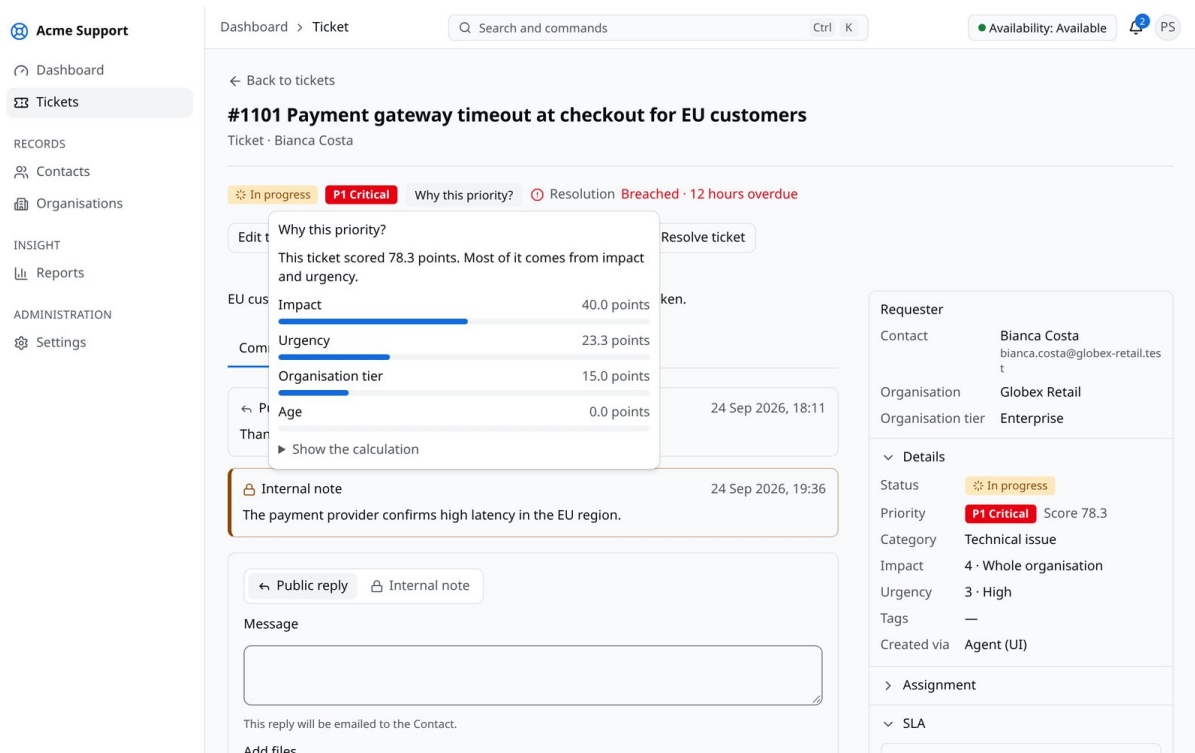


Figure 4.3: Priority explanation shown on a ticket

4.2 Testing

The system was tested manually in the testing phase. Each function was first tested on its own (unit testing), and complete workflows were then tested through the web application on the development machine and on the deployed server (system testing). Every test was run in both the light and the dark theme in Google Chrome, and tests were repeated after each correction.

4.2.1 Test Cases for Unit Testing

Table 4.2: Unit test cases

ID	Test case	Input	Expected output	Actual output	Result
UT-01	Sign in with correct details	registered email and password	dashboard opens	dashboard opened	Pass
UT-02	Sign in with a wrong password	registered email, wrong password	error message, no sign-in	"The workspace, email or password is incorrect." shown	Pass
UT-03	Create a contact	name and email	contact saved and listed	contact listed	Pass
UT-04	Create a contact without an email	name only	validation message	"Enter an email address." shown	Pass
UT-05	Edit an organisation	new customer tier	tier updated	tier updated	Pass
UT-06	Create a category	name and required skill	category saved	category listed	Pass
UT-07	Delete a category in use	category with tickets	deletion refused	category kept, error shown	Pass
UT-08	Create a ticket	title, description, contact, category	ticket saved with a number	ticket #1043 created	Pass
UT-09	Create a ticket without a title	empty title	validation message	"Enter a title." shown	Pass
UT-10	Add a public reply	reply text	reply shown in the conversation	reply shown	Pass
UT-11	Add an internal note	note text	note shown only to agents	note marked internal	Pass
UT-12	Attach a file	PNG image of 1 MB	file attached with a thumbnail	thumbnail shown	Pass

ID	Test case	Input	Expected output	Actual output	Result
UT-13	Invite a user	email and role	invitation email sent	invitation received	Pass
UT-14	Create a team	name and members	team saved	team listed	Pass
UT-15	Create an SLA policy	targets per priority	policy saved	policy listed	Pass

4.2.2 Test Cases for System Testing

Table 4.3: System test cases

ID	Scenario	Steps	Expected result	Result
ST-01	New user	owner invites a user; user accepts and signs in	user works with the invited role	Pass
ST-02	Ticket workflow	agent creates a ticket, replies, sets it pending, resolves and closes it	status changes shown; ticket closed	Pass
ST-03	Automatic handling	create a ticket in a category with eligible agents	priority, SLA timers and an assigned agent shown with explanations	Pass
ST-04	Duplicate suggestion	type a ticket similar to an existing one	existing ticket suggested before saving	Pass
ST-05	Email reply	contact replies to a notification email	reply appears on the ticket	Pass
ST-06	Workspace separation	user of workspace B opens a ticket link of workspace A	"not found" shown	Pass
ST-07	Report export	open a report and export CSV	file downloaded with the listed rows	Pass
ST-08	Deployment	open the live site and sign in	site served over HTTPS with the demonstration workspace	Pass

4.3 Result Analysis

All unit and system test cases passed. The main workflows worked end to end both on the development machine and on the deployed server, and the screens behaved the same in both themes.

The four algorithms were checked with worked examples calculated by hand. The priority level shown on each test ticket matched the hand calculation, and its explanation listed the share of each factor. New tickets went to the eligible agent with the fewest open tickets for their capacity, and agents without the required skill, off shift or at capacity were listed with the reason. Tickets typed with the same key words as an existing ticket were suggested as duplicates with the shared words shown, while unrelated tickets were not. SLA timers paused while a ticket waited for the customer, and the warning and breach notifications were sent at the expected times on working hours.

Reports showed the same numbers as the ticket list, and a ticket viewed at a past moment matched its earlier state. The limitations found were that duplicates written in completely different words are not detected, and that the behaviour under heavy load was not tested.

Chapter 5: Conclusion and Future Recommendations

5.1 Conclusion

This project built Smart Helpdesk, a multi-tenant support ticket management system in which one installation serves many organisations while keeping each organisation's data separate in the application and in the database. The system covers the full ticket workflow, a media library, email in both directions, a dashboard, reports with record history, a REST API and signed webhooks, and it runs on a cloud virtual machine or an organisation's own server.

All objectives of Section 1.3 were met. Four simple algorithms make the key decisions: a weighted priority score, least-loaded agent assignment, word-based duplicate detection and SLA timers on working hours, and each decision is explained to the agent. Manual unit and system testing confirmed that the features and the algorithms behave as specified. The main lessons were that explanations make automatic decisions easier to trust, that time handling is the hardest part of SLA management, and that data separation between organisations must be checked on every screen and API.

5.2 Future Recommendations

- **Load testing and monitoring** of the ticket list, ticket creation and dashboard under many users before wider use.
- **Stronger algorithms**, such as a priority score that considers the SLA deadline and duplicate detection that recognises different words with the same meaning.
- **A customer portal** where contacts follow their tickets, with live chat and messaging channels.
- **Weights learned from history**, fitting the priority and assignment settings from resolved tickets.
- **Mobile layouts** so that agents can work tickets from their phones.

References

- [1] Atlassian, "How impact and urgency are used to calculate priority," *Jira Service Management Cloud Documentation*. [Online]. Available: <https://support.atlassian.com/jira-service-management-cloud/docs/how-impact-and-urgency-are-used-to-calculate-priority/>
- [2] AXELOS, *Incident Management: ITIL 4 Practice Guide*. London, U.K.: AXELOS Limited, 2020.
- [3] "Simple additive weighting: a metamodel for multiple criteria decision analysis methods," *Expert Systems with Applications*, 2016, doi: 10.1016/j.eswa.2016.01.042.
- [4] N. Gans, G. Koole, and A. Mandelbaum, "Telephone call centers: Tutorial, review, and research prospects," *Manufacturing & Service Operations Management*, vol. 5, no. 2, pp. 79–141, 2003.
- [5] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008.
- [6] PostgreSQL Global Development Group, "Row security policies," *PostgreSQL 18 Documentation*. [Online]. Available: <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>
- [7] P. Runeson, M. Alexandersson, and O. Nyholm, "Detection of duplicate defect reports using natural language processing," in *Proc. 29th Int. Conf. Software Engineering*, 2007, pp. 499–510.
- [8] D. Harel, "Statecharts: A visual formalism for complex systems," *Science of Computer Programming*, vol. 8, no. 3, pp. 231–274, 1987.

Appendices

Appendix A: Screenshots

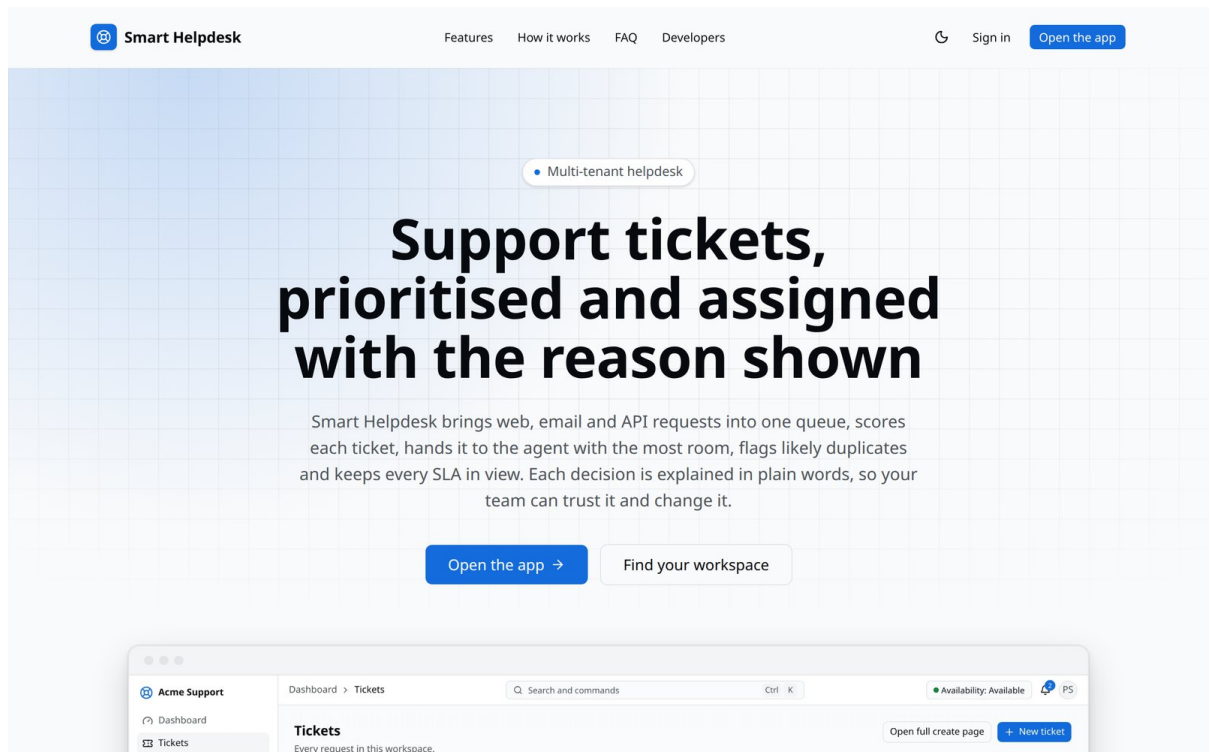


Figure A.1: Landing page

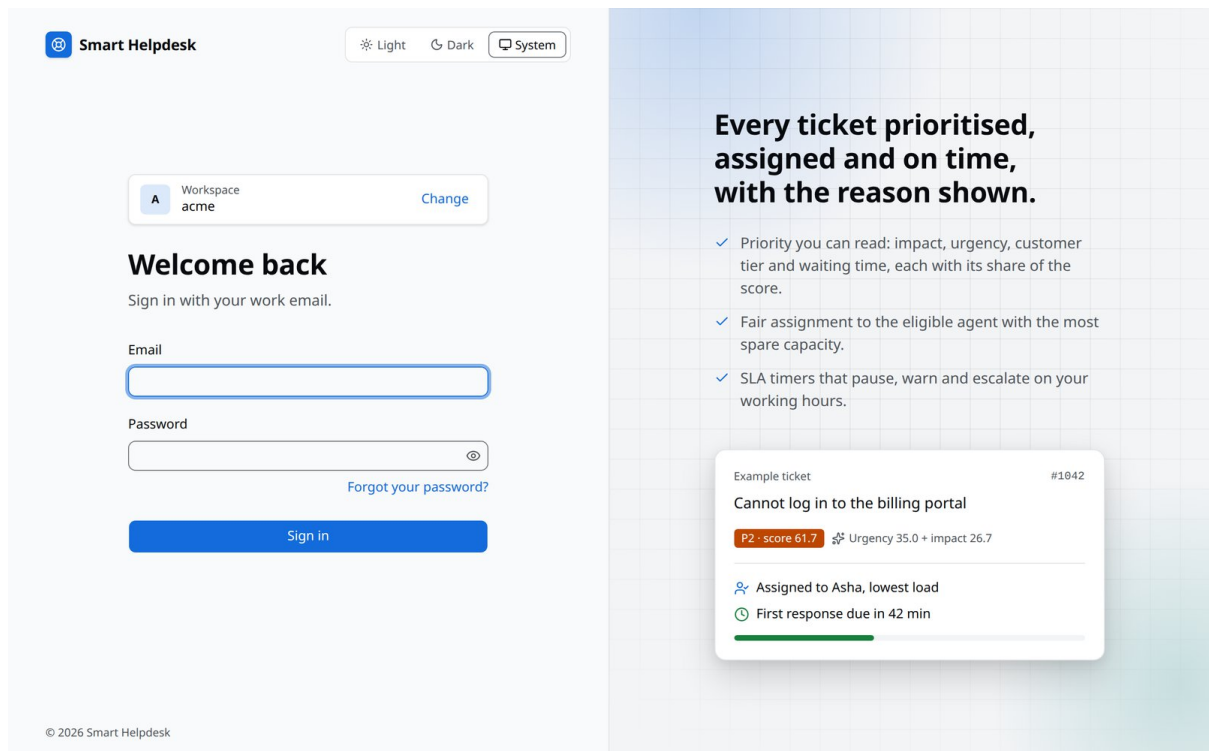


Figure A.2: Sign-in page of a workspace

Acme Support

Dashboard

Tickets

RECORDS

Contacts

Organisations

INSIGHT

Reports

ADMINISTRATION

Settings

Dashboard > Tickets

Search and commands

Ctrl K

Availability: Available

PS

Tickets

Open full create page

New ticket

Every request in this workspace.

All

My tickets

Unassigned

Breaching soon

Custom filters

Export CSV

Export XLSX

Search words or a ticket numb

Status 1

Priority

Assignee

Team

Category

Tag

Columns

Organisation

SLA

Duplicates: Any

Created

Clear filters

Status: Active (not resolved or closed) X

☐	Number ↑↓	Title	Channel	Status ↑↓	Priority ↓	SLA	Assignee	Age ↑↓
☐	#1101	Payment gateway timeout at c... Bianca Costa		In progress	P1 Critical	12 hours overdue	Deepa Karki	20 hours
☐	#1104	Cannot add new team membe... Nina Petrova		Assigned	P1 Critical	19 minutes left	Chen Wei	8 hours
☐	#1108	Search times out Victor Ahmed		Open	P2 High	23 hours left	Unassigned	1 hour
☐	#1118	DNS drops every few minutes Quentin Dubois		Open	P2 High	24 hours left	Unassigned	24 minutes
☐	#1112	Card charge is overdue Divya Nair		Open	P2 High	23 hours left	Unassigned	50 minutes
☐	#1106	Report builder is very slow Laura Schmidt		Open	P2 High	23 hours left	Unassigned	1 hour
☐	#1080	API returns 500 on POST /orde... Emil Novak		In progress	P2 High	13 hours left	Arjun Thapa	2 days
☐	#1055	VPN disconnects every 10 min... Mohan Das		In progress	P2 High	11 hours left	Deepa Karki	4 days

Figure A.3: Ticket list with filters

Reports

ADMINISTRATION

Settings

Description

After upgrading to v2.3.1 every POST /orders call returns a 500 error.

Contact

Search by name or email...

Create a contact

Category

Choose a category

Impact

Urgency

How much of the customer is affected.

How quickly harm grows.

Tags

Attachments

Add files

Choose Files

No file chosen

PNG, JPG, GIF, WebP, PDF, text, CSV, log, Office or ZIP. Up to 25 MB each.

Pick from library

Duplicates

#1080 API returns 500 on POST /orders since v2.3.1 91% similar
Shared words: 500, api, call, error, every, orders, post, returns, since, upgrading

#1081 POST /orders 500 error after upgrading to v2.3.1 43% similar
Shared words: 500, error, orders, post, since, upgrading

Create ticket

Figure A.4: New ticket with duplicate suggestions

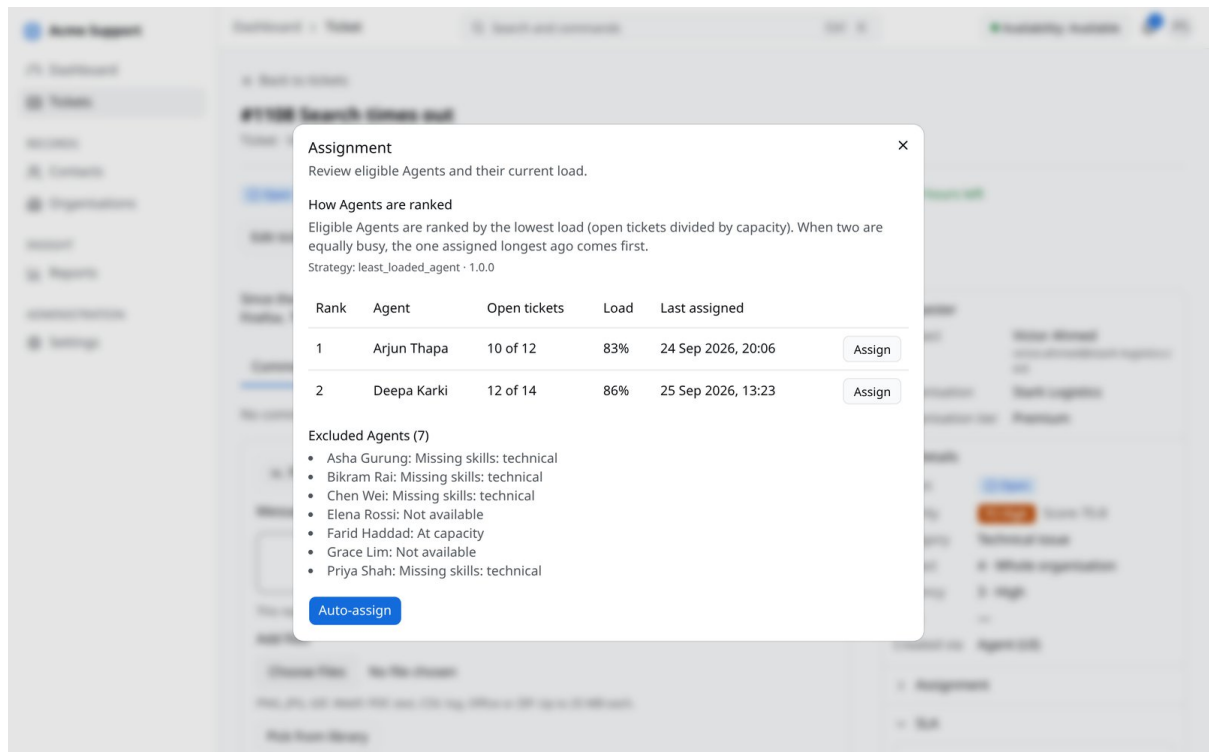


Figure A.5: Assignment ranking with excluded agents

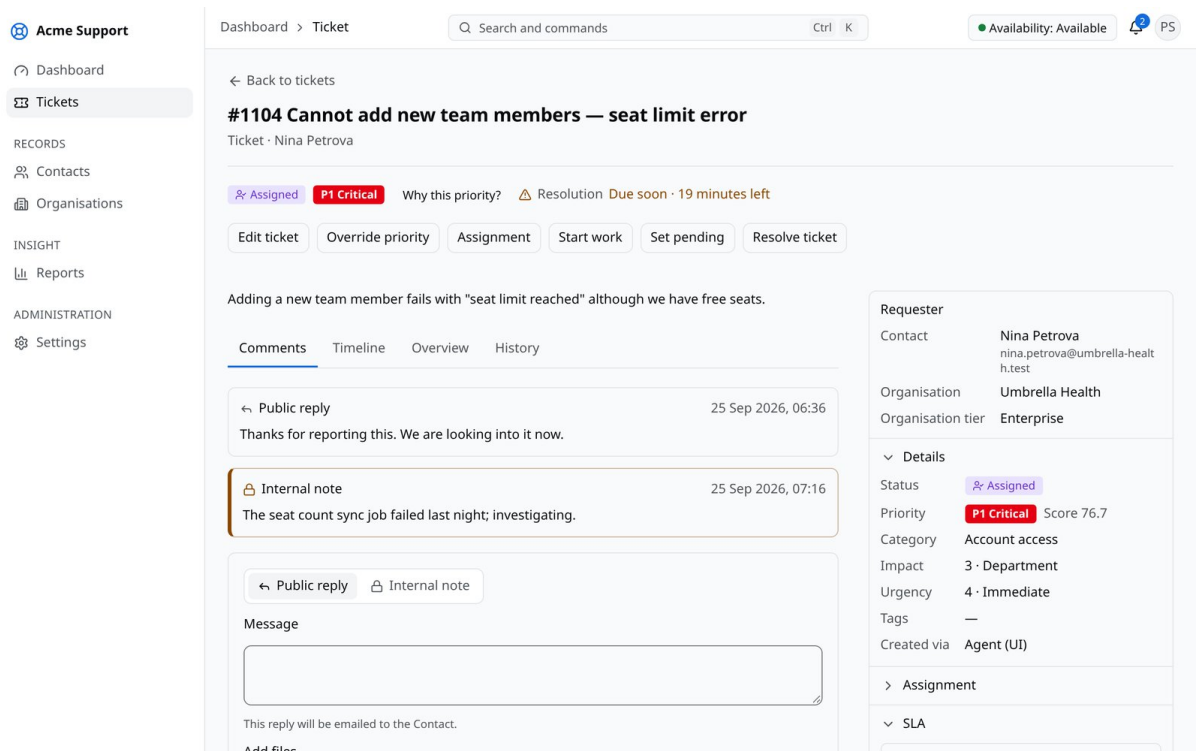


Figure A.6: Ticket close to its SLA deadline

Acme Support

Dashboard

Tickets

RECORDS

Contacts

Organisations

INSIGHT

Reports

ADMINISTRATION

Settings

Dashboard > Ticket

Search and commands

Ctrl K

Availability: Available

PS

Back to tickets

#1101 Payment gateway timeout at checkout for EU customers

Ticket · Bianca Costa

In progress

P1 Critical

Why this priority?

Resolution Breached · 12 hours overdue

Edit ticket

Override priority

Assignment

Set pending

Resolve ticket

EU customers get a gateway timeout at checkout; payments are not taken.

Comments

Timeline

Overview

History

As of

Pick a date and time in Asia/Kathmandu to see the record as it was then, and what has changed since.

Date and time

mm/dd/yyyy, --:--:-- --

Show this state

History

Every recorded change, newest first, with who made it.

Show: Changes and events

Event: sla breached by System

25 Sep 2026, 01:57:00

At set to 25 Sep 2026, 01:57:00

Kind set to resolution

Type set to breached

Details set to 1 values

Requester

Contact Bianca Costa

bianca.costa@globex-retail.test

Organisation Globex Retail

Organisation tier Enterprise

Details

Status In progress

Priority P1 Critical

Score 78.3

Category Technical issue

Impact 4 · Whole organisation

Urgency 3 · High

Tags

Created via Agent (UI)

Assignment

SLA

Figure A.7: Ticket history with the point-in-time view

Acme Support

Dashboard

Tickets

RECORDS

Contacts

Organisations

INSIGHT

Reports

ADMINISTRATION

Settings

Dashboard > Contacts

Search and commands

Ctrl K

Availability: Available

PS

Contacts

The people who write to this workspace.

+ New contact

Search by name or email...

Organisation

Tag

Archived: Hide archived

Columns

☐	Name ↑	Email ↑	Phone	Organisation	Tags	Last ticket ↑	Added ↑
☐	Aarav Sharma	aarav.sharma@globex-retail.test	—	Globex Retail	—	23 Sep 2026	27 Jun 2026
☐	Anil Shrestha	anil.shrestha@wayne-foods.test	—	Wayne Foods	—	24 Sep 2026	27 Jun 2026
☐	Beatriz Silva	beatriz.silva@wayne-foods.test	—	Wayne Foods	—	24 Sep 2026	27 Jun 2026
☐	Bianca Costa	bianca.costa@globex-retail.test	—	Globex Retail	—	25 Sep 2026	27 Jun 2026
☐	Carlos Mendez	carlos.mendez@globex-retail.test	—	Globex Retail	—	22 Sep 2026	27 Jun 2026
☐	Chloe Martin	chloe.martin@wayne-foods.test	—	Wayne Foods	—	25 Sep 2026	27 Jun 2026
☐	Daniel Kim	daniel.kim@wayne-foods.test	—	Wayne Foods	—	23 Sep 2026	27 Jun 2026
☐	Divya Nair	divya.nair@globex-retail.test	—	Globex Retail	—	25 Sep 2026	27 Jun 2026
☐	Emil Novak	emil.novak@globex-retail.test	—	Globex Retail	—	23 Sep 2026	27 Jun 2026
☐	Ethan Brooks	ethan.brooks@example.test	—	—	—	22 Sep 2026	27 Jun 2026
☐	Fatima Khan	fatima.khan@initech.test	—	Initech	—	25 Sep 2026	27 Jun 2026
☐	Freya Olsen	freya.olsen@example.test	—	—	—	24 Sep 2026	27 Jun 2026
☐	George Miller	george.miller@initech.test	—	Initech	—	19 Sep 2026	27 Jun 2026

Rows per page 25

1–25 of 33

Page 1 of 2

Figure A.8: Contact list

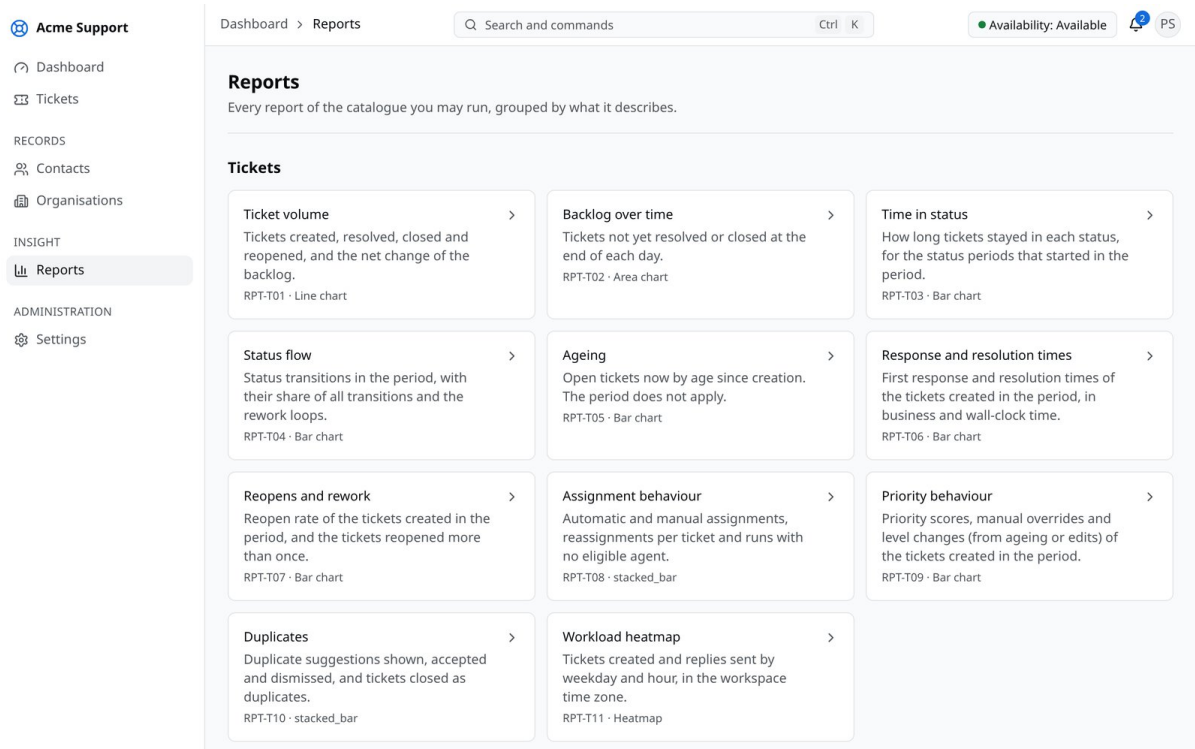


Figure A.9: Report catalogue

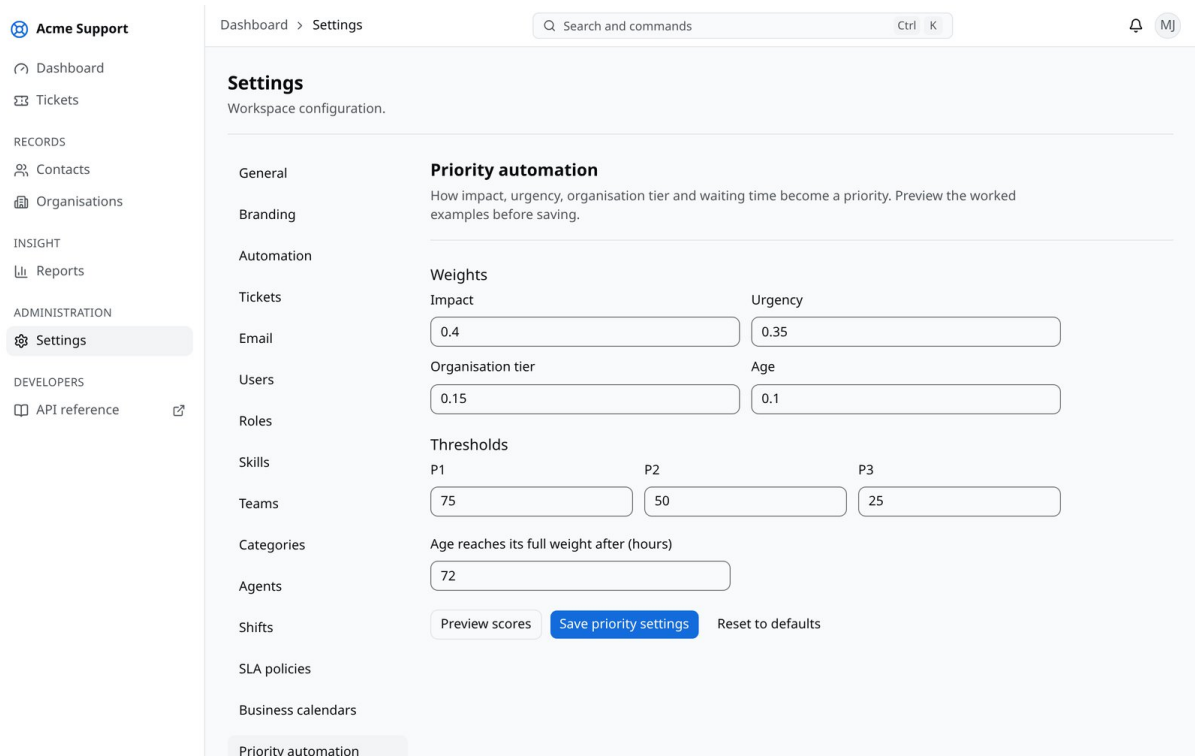


Figure A.10: Priority settings

Acme Support

Dashboard

Tickets

RECORDS

Contacts

Organisations

INSIGHT

Reports

ADMINISTRATION

Settings

Dashboard > Settings

Search and commandsCtrl K

Availability: AvailablePS

Settings

Workspace configuration.

Skills

Teams

Categories

Agents

Shifts

SLA policies

Business calendars

Media library

SLA policies

The response and resolution times promised per priority. The time counts only the working hours of the chosen Business calendar.

Add policy

Default

Edit

Applies to

Default

Counts time on

24x7

Warns at

75% of the target

Version

2

Priority	First response	Resolution
P1 Critical	30 min	8 h
P2 High	2 h	24 h
P3 Medium	8 h	72 h
P4 Low	24 h	168 h

Figure A.11: SLA policies

Acme Support

Dashboard

Tickets

RECORDS

Contacts

Organisations

INSIGHT

Reports

ADMINISTRATION

Settings

Dashboard > Tickets

Search and commandsCtrl K

Availability: AvailablePS

Tickets

Every request in this workspace.

Open full create page

New ticket

AllMy ticketsUnassignedBreaching soonCustom filters

Export CSV

Export XLSX

Search words or a ticket numb

Status 1

Priority

Assignee

Team

Category

Tag

Columns

Organisation

SLA

Duplicates: Any

Created

Clear filters

Status: Active (not resolved or closed) x

☐	Number ↕	Title	Channel	Status ↕	Priority ↕	SLA	Assignee	Age ↕
☐	#1101	Payment gateway timeout at c... Bianca Costa	🖥️	In progress	P1 Critical	🕒 12 hours overdue	Deepa Karki	20 hours
☐	#1104	Cannot add new team membe... Nina Petrova	🖥️	Assigned	P1 Critical	⚠️ 18 minutes left	Chen Wei	8 hours
☐	#1108	Search times out Victor Ahmed	🖥️	Open	P2 High	🕒 23 hours left	Unassigned	1 hour
☐	#1118	DNS drops every few minutes Quentin Dubois	🌐	Open	P2 High	🕒 24 hours left	Unassigned	25 minutes
☐	#1112	Card charge is overdue Divya Nair	🖥️	Open	P2 High	🕒 23 hours left	Unassigned	51 minutes
☐	#1106	Report builder is very slow Laura Schmidt	🖥️	Open	P2 High	🕒 23 hours left	Unassigned	1 hour
☐	#1080	API returns 500 on POST /orde... Emil Novak	🖥️	In progress	P2 High	🕒 13 hours left	Arjun Thapa	2 days
☐	#1055	VPN disconnects every 10 min... Mohan Das	🖥️	In progress	P2 High	🕒 11 hours left	Deepa Karki	4 days

Figure A.12: Ticket list in the dark theme

44

Platform administration

Platform docs PA

Tenants

Every workspace on this installation.

Name	Workspace	Status	Plan	Created
Acme Support	acme	Active	Standard	27 Jun 2026
Globex Retail Support	globex	Active	Standard	27 Jul 2026

Figure A.13: Platform console with the list of workspaces